



IASI Quarto User Guide

Framework for Engineering Documentation with Quarto

IASI Project

Table of contents

1	Introducción	7
1.1	iasi.quarto y Quarto	7
1.2	¿Qué encontrarás en este libro?	7
1.3	Requisitos	8
2	Quarto como base	9
2.1	Qué hace Quarto	10
2.2	Qué añade iasi.quarto	11
2.3	Separación de responsabilidades	12
2.4	Un proyecto sigue siendo un proyecto Quarto	13
2.5	Cuándo resulta útil	14
3	Qué es iasi.quarto	15
3.1	Una convención sobre Quarto	15
3.2	El problema que resuelve	15
3.3	Qué aporta	16
3.4	Qué no pretende ser	16
3.5	Una herramienta para proyectos que crecen	17
4	Fundamentos	18
4.1	La publicación IASI Quarto	18
4.1.1	La firma de una publicación	18
4.1.2	Una publicación no es necesariamente un libro	19
4.1.3	Publicación individual	20
4.1.4	Multiproyecto	20
4.1.5	La publicación actual tiene prioridad	21
4.1.6	Publicación y contenido	22
4.1.7	Archivos fuente y archivos derivados	23
4.1.8	Reconocer no significa construir	24
4.1.9	La publicación como unidad de construcción	24
4.2	Estructura de un proyecto	25
4.3	Configuración general y configuración por formato	26
4.4	Configuración de iasi.quarto	27
4.5	Directorio de contenido	28
4.6	Documentos de raíz	28

4.7	Carpetas de contenido	29
4.8	Estructura regular	29
4.9	Estructura structured	31
4.10	Archivos fuente y archivos derivados	32
4.11	La estructura preparada	32
4.12	Publicaciones múltiples	33
4.13	Una estructura explícita	33
4.14	El archivo <code>_iasi.yml</code>	34
4.15	Valores predeterminados	34
4.16	strategy	35
	4.16.1 regular	35
	4.16.2 structured	36
	4.16.3 direct	36
4.17	content-dir	37
4.18	numbered	37
	4.18.1 numbered: false	38
4.19	Configuración parcial	39
4.20	Configuración válida completa	40
4.21	YAML mal formado	40
4.22	Valores semánticamente inválidos	41
4.23	Propiedades desconocidas	42
4.24	Separación de responsabilidades	42
4.25	Configuración y ciclo de construcción	43
4.26	Un contrato pequeño	44
4.27	Estrategias de publicación	44
4.28	Una misma estructura física, distintas publicaciones	45
4.29	Estrategia regular	46
	4.29.1 Propiedad del <code>index.qmd</code>	47
	4.29.2 <code>front-matter.quarto</code>	47
	4.29.3 Estructura resultante	48
	4.29.4 Cuándo utilizar regular	49
4.30	Estrategia structured	49
	4.30.1 Propiedad del <code>index.qmd</code>	50
	4.30.2 Estructura resultante	51
	4.30.3 <code>front-matter.quarto</code> no participa	51
	4.30.4 Cuándo utilizar structured	52
4.31	Estrategia direct	52
	4.31.1 Qué no hace direct	53
	4.31.2 Cuándo utilizar direct	53
4.32	Comparación	54
4.33	Estrategia y numeración	54
4.34	Estrategia y formatos de salida	55
4.35	Elección de estrategia	56

4.36	Una decisión editorial explícita	56
4.37	Ciclo de construcción	57
4.38	<code>build()</code> como orquestador	57
4.39	Primera etapa: <code>validate()</code>	58
4.39.1	Publicación actual	59
4.39.2	Multiproyecto	60
4.40	Selección de publicaciones	60
4.41	Segunda etapa: <code>discover()</code>	61
4.41.1	Aplicación de valores predeterminados	62
4.42	Tercera etapa: <code>check()</code>	63
4.42.1	Resultado de la comprobación	63
4.43	Cuarta etapa: <code>prepare()</code>	64
4.43.1	Preparación <code>regular</code>	64
4.43.2	Preparación <code>structured</code>	65
4.43.3	Preparación <code>direct</code>	65
4.43.4	Artefactos	65
4.44	Idempotencia de la preparación	65
4.45	Resolución de formatos	66
4.46	Quinta etapa: <code>render()</code>	67
4.46.1	Delegación en Quarto	68
4.46.2	Resultado	68
4.47	Fallar antes de renderizar	68
4.48	Ciclo de una publicación individual	69
4.49	Ciclo de un multiproyecto	70
4.50	Estado acumulado	71
4.51	Un pipeline deliberado	71
5	Instalación	73
5.1	Instalación	73
5.2	Requisitos	74
5.3	R	74
5.4	Quarto	75
5.5	HTML	76
5.6	PDF	76
5.7	Git	77
5.8	Acceso al repositorio	77
5.9	Permisos de escritura	78
5.10	Red y servicios externos	78
5.11	Comprobación mínima	78
5.12	Instalar <code>iasi.quarto</code>	79
5.13	Instalar <code>remotes</code>	80
5.14	Instalar la versión actual	80
5.15	Instalar una versión concreta	81

5.16	Actualizar el paquete	81
5.17	Comprobar la versión instalada	82
5.18	Biblioteca de instalación	83
5.19	Dependencias	84
5.20	Instalación desde una copia local	84
5.21	Cargar el paquete	85
5.22	Instalación reproducible	85
5.23	Instalación mínima	86
5.24	Verificar la instalación	86
5.25	Comprobar el paquete	86
5.26	Comprobar Quarto	87
5.27	Situarse en una publicación	88
5.28	Validar la publicación	88
5.28.1	Publicación actual	89
5.28.2	Multiproyecto	89
5.29	Directorio no reconocido	90
5.30	Construir HTML	91
5.31	Inspeccionar el resultado	91
5.32	Construir PDF	92
5.33	Verificación completa	93
5.34	Diagnóstico rápido	94
5.35	Resultado esperado	94
6	Workflow recomendado	96
6.1	1. Trabajar desde la raíz adecuada	96
6.2	2. Validar el espacio de trabajo	96
6.3	3. Construir HTML durante la edición	97
6.4	4. Construir todos los formatos	97
6.5	5. Revisar los artefactos construidos	98
6.6	6. Ensamblar <code>publish/</code>	98
6.7	7. Utilizar <code>deploy()</code> al cerrar el ciclo	98
6.8	Ciclo resumido	99
7	API pública	100
7.1	<code>validate()</code>	100
7.2	Resultado de <code>validate()</code>	101
7.2.1	<code>plan\$path</code>	101
7.2.2	<code>plan\$current</code>	102
7.2.3	<code>plan\$books</code>	102
7.3	Ruta no reconocida	102
7.4	Qué hace <code>validate()</code>	103
7.5	Ejemplo con una publicación	103
7.6	Ejemplo con un multiproyecto	104

7.7	<code>build()</code>	104
7.8	Construcción predeterminada	105
7.9	Argumento <code>path</code>	106
7.10	Argumento <code>format</code>	106
7.11	Argumento <code>book</code>	108
	7.11.1 Nombre completo	108
	7.11.2 Nombre sin prefijo numérico	108
	7.11.3 Prefijo numérico	108
7.12	<code>book = NULL</code> y <code>book = "all"</code>	109
7.13	Selecciones no encontradas	109
7.14	Selección antes del descubrimiento	109
7.15	Ejemplos de uso	110
	7.15.1 Construir la publicación actual	110
	7.15.2 Construir solo HTML	110
	7.15.3 Construir solo PDF	110
	7.15.4 Construir un multiproyecto completo	110
	7.15.5 Construir una publicación del multiproyecto	111
	7.15.6 Construir una publicación y un formato	111
	7.15.7 Construir varias publicaciones	111
7.16	Resultado de <code>build()</code>	111
7.17	Mensajes de construcción	112
7.18	<code>publish()</code>	113
7.19	<code>deploy()</code>	114
7.20	API pública y funciones privadas	114
7.21	Por qué no se expone cada etapa	115
7.22	Elegir la operación pública	115
7.23	Llamadas con argumentos nombrados	116
7.24	Contrato estable, implementación evolutiva	117

1 Introducción

Bienvenido a la guía de usuario de **iasi.quarto**.

Este libro está dirigido a quienes desean utilizar **iasi.quarto** para construir proyectos documentales basados en Quarto.

No es necesario conocer la arquitectura interna del framework ni los detalles de su implementación.

El objetivo de esta guía es ayudarte a comprender la filosofía de **iasi.quarto**, instalarlo, crear tu primer proyecto y aprovechar sus capacidades en situaciones reales.

1.1 iasi.quarto y Quarto

iasi.quarto vive sobre Quarto.

No pretende sustituirlo, modificar su modelo documental ni crear un sistema de publicación alternativo.

Quarto continúa siendo el motor responsable de procesar los documentos y generar los formatos finales. **iasi.quarto** proporciona una capa de organización, automatización y convenciones que facilita el desarrollo y mantenimiento de proyectos documentales de cualquier tamaño.

1.2 ¿Qué encontrarás en este libro?

A lo largo de esta guía aprenderás a:

- comprender el propósito de **iasi.quarto**;
- instalar el framework;
- crear un nuevo proyecto;
- organizar la documentación;
- generar libros en distintos formatos;
- configurar el comportamiento del framework;
- seguir un workflow recomendado desde la edición hasta la publicación;
- utilizar la API pública;
- resolver los problemas más habituales.

Cada capítulo desarrolla un aspecto concreto y puede consultarse de forma independiente cuando ya se conoce el funcionamiento básico del framework.

1.3 Requisitos

Se recomienda tener conocimientos básicos de:

- Quarto;
- Git;
- R (para utilizar el paquete `iasi.quarto`).

No obstante, los conceptos específicos del framework se explican desde el principio.

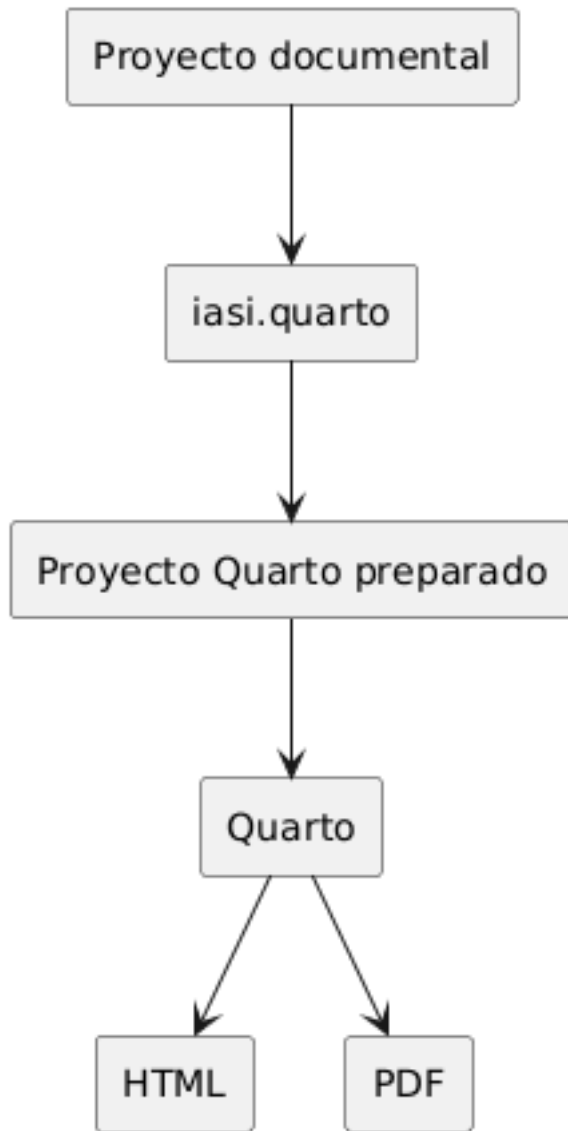
Comencemos entendiendo por qué nació el framework y qué problemas pretende resolver.

2 Quarto como base

`iasi.quarto` está construido sobre Quarto.

No introduce un nuevo formato documental ni sustituye el proceso de publicación de Quarto. Los archivos continúan siendo documentos `.qmd`, la configuración sigue expresándose mediante YAML y los formatos finales son generados por Quarto.

Esta relación permite utilizar `iasi.quarto` sin abandonar el ecosistema original.



La idea central es sencilla:

`iasi.quarto` organiza y prepara. Quarto renderiza.

2.1 Qué hace Quarto

Quarto es el motor de publicación.

Entre otras tareas, se encarga de:

- interpretar los documentos `.qmd`;
- aplicar la configuración definida en `_quarto.yml`;
- ejecutar el código incluido en los documentos;
- resolver referencias, citas, figuras y tablas;
- aplicar temas, plantillas y estilos;
- generar formatos como HTML y PDF.

Un proyecto utilizado por `iasi.quarto` continúa siendo un proyecto Quarto y puede renderizarse directamente con sus herramientas habituales:

```
quarto render
```

`iasi.quarto` no altera ese principio.

2.2 Qué añade `iasi.quarto`

En un libro Quarto, la estructura de la publicación se declara normalmente en `_quarto.yml`.

Por ejemplo:

```
book:
  chapters:
    - index.qmd
    - chapters/01-intro.qmd
    - chapters/02-installation.qmd
    - chapters/03-usage.qmd
```

Esta configuración es clara y suficiente para publicaciones pequeñas. Sin embargo, a medida que un proyecto crece, mantener manualmente la relación de capítulos, partes y documentos puede convertirse en una tarea repetitiva y propensa a errores.

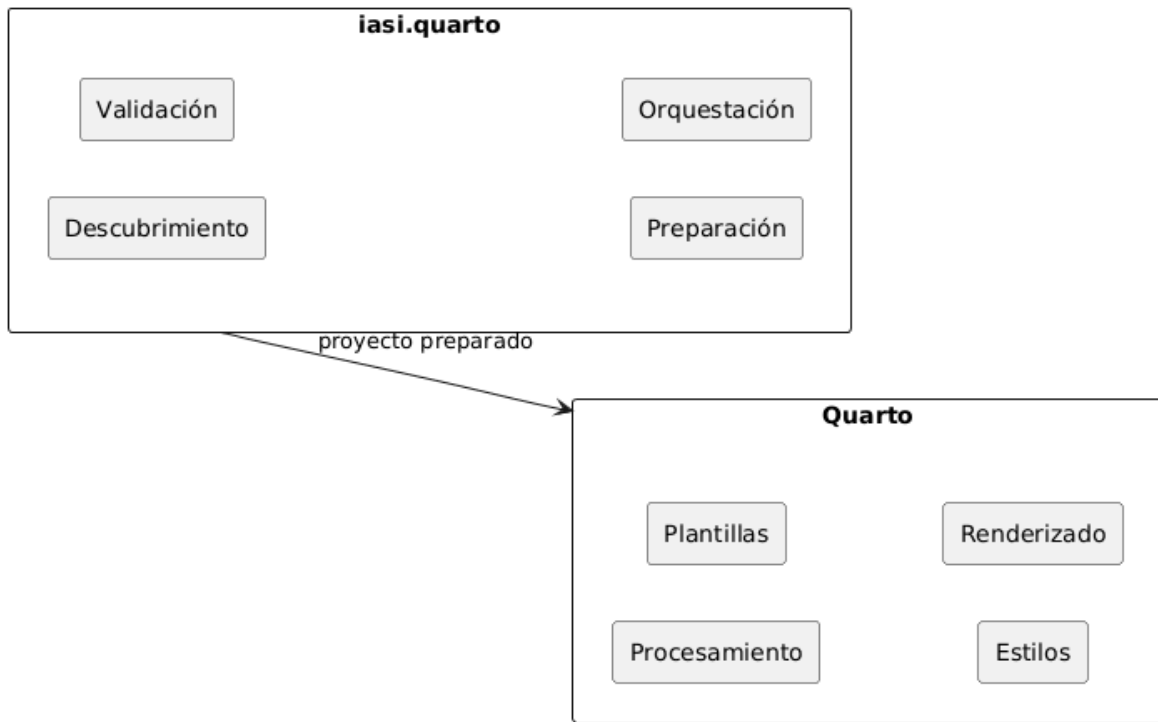
`iasi.quarto` añade una capa de convenciones y automatización que permite deducir parte de esa estructura a partir de la organización del proyecto.

Su trabajo consiste en:

- descubrir los documentos que forman la publicación;
- interpretar su organización;
- comprobar que la estructura sea coherente;
- preparar la configuración que necesita Quarto;
- coordinar la generación de uno o varios formatos.

2.3 Separación de responsabilidades

La separación entre ambas herramientas es deliberada.



Quarto controla:

- el modelo documental;
- los formatos de salida;
- los motores de ejecución;
- las plantillas;
- los estilos;
- el proceso de renderizado.

`iasi.quarto` controla:

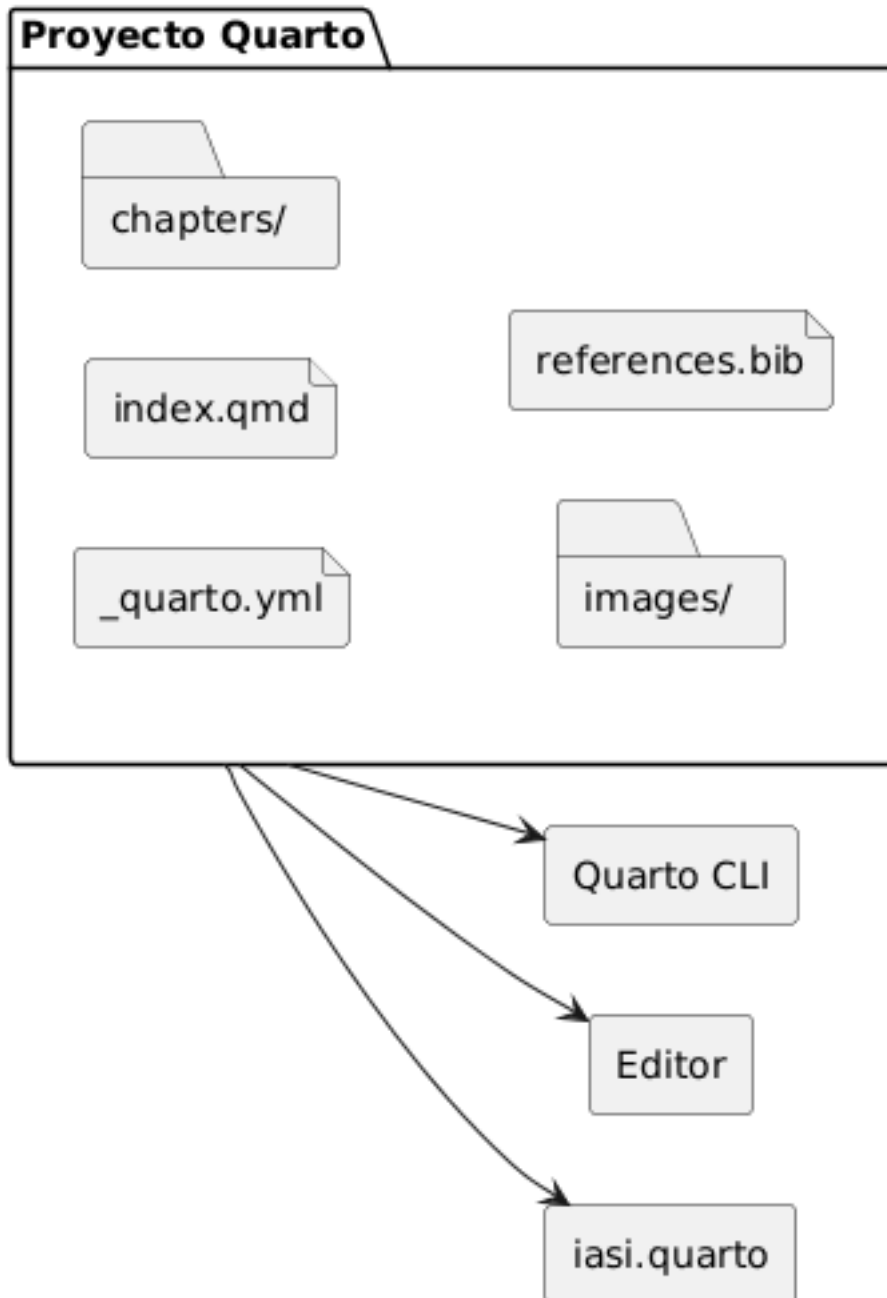
- las convenciones de organización;
- el descubrimiento de contenido;
- la validación de la estructura;
- la preparación de la publicación;
- la coordinación de libros y formatos.

Esta división evita duplicar capacidades que Quarto ya resuelve correctamente.

2.4 Un proyecto sigue siendo un proyecto Quarto

Adoptar `iasi.quarto` no encierra la publicación en un formato propietario.

El resultado preparado continúa utilizando archivos reconocibles por Quarto:



Esto permite:

- utilizar directamente la línea de comandos de Quarto;
- trabajar con las extensiones habituales del editor;
- conservar temas y configuraciones existentes;
- integrar otras extensiones de Quarto;
- dejar de utilizar `iasi.quarto` sin convertir los documentos.

`iasi.quarto` añade una capa sobre el proyecto, pero no se apropia de él.

2.5 Cuándo resulta útil

La automatización aporta más valor cuando:

- la publicación contiene muchos capítulos;
- los documentos cambian de posición con frecuencia;
- existen varios libros en un mismo repositorio;
- deben generarse varios formatos;
- se desea aplicar una organización uniforme;
- el proyecto debe poder crecer sin mantener manualmente listas extensas.

En publicaciones pequeñas, la ventaja principal es la sencillez.

En publicaciones grandes, la ventaja es mantener bajo control una estructura que, de otro modo, acabaría dispersa entre directorios, nombres de archivos y configuración YAML.

En el siguiente capítulo veremos cómo instalar `iasi.quarto` y comprobar que Quarto y R están correctamente disponibles.

3 Qué es `iasi.quarto`

`iasi.quarto` es una capa de organización y automatización construida sobre Quarto.

Su propósito no es sustituir el modelo de publicación de Quarto, sino facilitar el trabajo cuando un proyecto empieza a crecer: aparecen más capítulos, más formatos, distintas estructuras documentales y la necesidad de repetir tareas de preparación y renderizado de forma predecible.

3.1 Una convención sobre Quarto

Quarto ya sabe transformar documentos en libros, sitios web, artículos y otros formatos. `iasi.quarto` se ocupa de lo que ocurre alrededor de ese proceso:

- descubrir la estructura del proyecto;
- validar que esa estructura sea coherente;
- preparar los artefactos necesarios;
- renderizar uno o varios formatos;
- coordinar la construcción de uno o varios libros.

Quarto continúa siendo el motor de publicación. `iasi.quarto` actúa como capa de ingeniería.

3.2 El problema que resuelve

En un proyecto pequeño, mantener manualmente `_quarto.yml` puede ser suficiente. Cuando el número de documentos aumenta, esa configuración empieza a concentrar demasiado conocimiento:

- qué archivos forman parte del libro;
- en qué orden deben aparecer;
- cómo se organizan los capítulos;
- qué perfiles existen;
- qué formato debe generarse;
- dónde se escriben los resultados.

`iasi.quarto` convierte parte de ese conocimiento implícito en convenciones verificables.

La estructura del proyecto deja de ser una colección de decisiones manuales y pasa a ser información que el framework puede descubrir, validar y preparar.

3.3 Qué aporta

El flujo fundamental de `iasi.quarto` se divide en cuatro pasos:

```
project = discover()
validation = validate(project)
publication = prepare(validation)
render(publication)
```

En el uso habitual, `build()` coordina ese proceso completo.

Cada etapa tiene una responsabilidad concreta:

- `discover()` interpreta la estructura existente;
- `validate()` comprueba que el proyecto sea coherente;
- `prepare()` genera la configuración derivada necesaria;
- `render()` delega en Quarto la producción de los formatos finales.

Esta separación permite detectar los problemas antes de iniciar un renderizado y hace que cada transformación sea explícita.

3.4 Qué no pretende ser

`iasi.quarto` no es:

- un sustituto de Quarto;
- un nuevo lenguaje documental;
- un generador de contenido;
- una plantilla cerrada;
- una abstracción que oculte por completo la configuración de Quarto.

El proyecto sigue siendo un proyecto Quarto normal. Sus archivos pueden abrirse, editarse y renderizarse con las herramientas habituales.

La diferencia es que ciertas decisiones estructurales se expresan mediante convenciones que `iasi.quarto` puede comprender.

3.5 Una herramienta para proyectos que crecen

El valor de `iasi.quarto` aparece cuando la documentación deja de ser un conjunto de archivos aislados y empieza a comportarse como un sistema.

En ese momento importan el orden, la consistencia, la repetibilidad y la capacidad de modificar la estructura sin reconstruir manualmente toda la configuración.

`iasi.quarto` proporciona ese armazón sin quitarle a Quarto el papel central que le corresponde.

4 Fundamentos

4.1 La publicación IASI Quarto

Una **publicación IASI Quarto** es un proyecto Quarto cuya estructura, configuración y proceso de construcción siguen las convenciones definidas por `iasi.quarto`.

No es un tipo diferente de documento. Sigue siendo una publicación Quarto y puede utilizar sus formatos, extensiones y mecanismos habituales. `iasi.quarto` añade una capa de organización y automatización sobre ese proyecto:

- reconoce publicaciones;
- interpreta su estructura;
- comprueba su coherencia;
- genera los artefactos derivados;
- coordina su construcción.

La publicación continúa perteneciendo a Quarto. `iasi.quarto` no sustituye su motor, sino que organiza todo lo que ocurre antes de invocarlo.

4.1.1 La firma de una publicación

Una carpeta se reconoce como publicación IASI Quarto cuando contiene estos cuatro archivos:

```
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
_iasi.yml
```

Los cuatro forman la **firma de la publicación**.

`_quarto.yml` contiene la configuración común del proyecto Quarto.

`_quarto-html.yml` contiene la configuración específica del perfil HTML.

`_quarto-pdf.yml` contiene la configuración específica del perfil PDF.

`_iasi.yml` declara cómo debe interpretar y preparar la publicación `iasi.quarto`.

La coexistencia de los cuatro archivos es intencionada. La presencia aislada de `_quarto.yml` únicamente identifica un proyecto Quarto. La presencia aislada de `_iasi.yml` tampoco es suficiente. Solo la firma completa identifica una publicación IASI Quarto.

Una estructura mínima puede ser:

```
my-publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  _iasi.yml  
  index.qmd  
  chapters/
```

La firma permite reconocer el proyecto sin analizar todavía toda su configuración ni recorrer su contenido. El reconocimiento es deliberadamente superficial: primero se determina qué existe; después se estudia qué contiene.

4.1.2 Una publicación no es necesariamente un libro

`iasi.quarto` utiliza el término **publicación** para referirse a la unidad que puede comprobarse, prepararse y construirse.

Actualmente, el caso principal es un libro Quarto:

```
project:  
  type: book
```

Sin embargo, la publicación y el tipo de proyecto son conceptos diferentes.

La publicación es la unidad gestionada por `iasi.quarto`.

El tipo indica cómo entiende Quarto ese proyecto:

```
book  
website  
manuscript  
article
```

Esta separación evita incorporar en la arquitectura una equivalencia innecesaria:

```
publicación libro
```

Un libro es un tipo posible de publicación. La publicación es el concepto más general.

4.1.3 Publicación individual

Cuando la carpeta indicada contiene directamente la firma completa, se considera una **publicación individual**.

Por ejemplo:

```
user-guide/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  _iasi.yml  
  index.qmd  
  chapters/
```

En este caso, la carpeta `user-guide/` es la raíz de la publicación.

La construcción se realiza desde esa raíz y todos los archivos de configuración, contenido y salida se interpretan en relación con ella.

```
validate("user-guide")
```

También puede validarse desde el propio directorio:

```
validate()
```

4.1.4 Multiproyecto

Una carpeta puede actuar como contenedor de varias publicaciones independientes.

```
docs/  
  user-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    _iasi.yml
```

```
index.qmd
chapters/

developer-guide/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  _iasi.yml
  index.qmd
  chapters/
```

La carpeta `docs/` no es una publicación. Es un **multiproyecto** que contiene dos publicaciones:

```
user-guide
developer-guide
```

Cada una conserva su propia configuración, estrategia, contenido y perfiles de salida.

El multiproyecto permite construirlas conjuntamente sin convertirlas en partes de un único libro.

```
validate("docs")
```

La validación reconoce el contenedor y localiza las publicaciones que contiene.

La raíz del multiproyecto no necesita disponer de su propio `_iasi.yml`. Su función es agrupar publicaciones completas, no comportarse como una publicación adicional.

4.1.5 La publicación actual tiene prioridad

Cuando la carpeta indicada contiene la firma completa, se interpreta como una publicación individual aunque existan otras firmas en directorios descendientes.

```
publication/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  _iasi.yml
  index.qmd
  chapters/
```

```
examples/  
  sample-book/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    _iasi.yml
```

En este caso, `publication/` es la publicación actual. La publicación situada bajo `examples/` no se incorpora automáticamente al mismo plan de construcción.

Esta regla evita que ejemplos, plantillas, pruebas o proyectos auxiliares se confundan con publicaciones hermanas.

El reconocimiento sigue este orden:

1. comprobar si la carpeta actual contiene la firma completa;
2. si la contiene, tratarla como una publicación individual;
3. si no la contiene, buscar publicaciones completas en sus descendientes;
4. si se encuentran, tratar la carpeta como multiproyecto;
5. si no se encuentra ninguna, concluir que no es un proyecto IASI Quarto.

4.1.6 Publicación y contenido

La raíz de la publicación no obliga a utilizar un directorio llamado `chapters`.

El directorio de contenido se declara en `_iasi.yml`:

```
publication:  
  content-dir: chapters
```

Podría utilizarse otro nombre:

```
publication:  
  content-dir: content
```

Y la estructura correspondiente sería:

```
my-publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  _iasi.yml
```

```
index.qmd
content/
```

La publicación es la raíz completa del proyecto. El directorio de contenido es solo una de sus partes.

Esta distinción permite que una publicación contenga, además del texto principal:

```
images/
resources/
bibliography/
extensions/
scripts/
outputs/
```

`iasi.quarto` no redefine toda la estructura de Quarto. Solo necesita conocer dónde se encuentra el contenido que debe organizar.

4.1.7 Archivos fuente y archivos derivados

Dentro de una publicación existen dos clases de archivos.

Los **archivos fuente** pertenecen al autor:

```
index.qmd
_iasi.yml
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
chapters/01-intro/01-introduction.qmd
```

Los **archivos derivados** son generados por `iasi.quarto` durante la preparación:

```
_book-structure.yml
chapters/01-intro/index.qmd
```

La lista exacta depende de la estrategia de publicación.

La diferencia es importante:

- los archivos fuente expresan decisiones editoriales;
- los archivos derivados materializan esas decisiones para Quarto.

Un archivo derivado puede regenerarse. No debe convertirse accidentalmente en la única copia de contenido escrito por el autor.

4.1.8 Reconocer no significa construir

El reconocimiento de una publicación no implica que sea válida ni que pueda construirse.

Una carpeta puede contener la firma completa y, aun así, presentar problemas:

```
_iasi.yml mal formado
estrategia desconocida
directorio de contenido inexistente
index.qmd raíz ausente
estructura incompatible con la estrategia
```

Por eso `validate()` responde a una pregunta limitada:

¿Existe aquí una publicación IASI Quarto o un multiproyecto reconocible?

No genera archivos, no modifica el proyecto y no ejecuta Quarto.

```
validate()
```

La construcción completa corresponde a:

```
build()
```

Esta separación permite inspeccionar un proyecto antes de intervenir sobre él.

4.1.9 La publicación como unidad de construcción

Una publicación reúne todo lo necesario para ejecutar una construcción independiente:

```
configuración común
configuración por formato
declaración IASI
contenido
artefactos derivados
salidas
```

En un multiproyecto, cada publicación mantiene esa independencia.

Esto permite construir:

- todas las publicaciones;
- una publicación concreta;
- todos los formatos;
- un formato determinado.

Por ejemplo:

```
build(  
  book = "user-guide",  
  format = "html"  
)
```

La selección pertenece al proceso de construcción. La publicación, por su parte, continúa siendo una unidad completa y autosuficiente.

Esa es la frontera fundamental del modelo:

Una publicación IASI Quarto es una unidad Quarto reconocible, configurable y construible de forma independiente.

4.2 Estructura de un proyecto

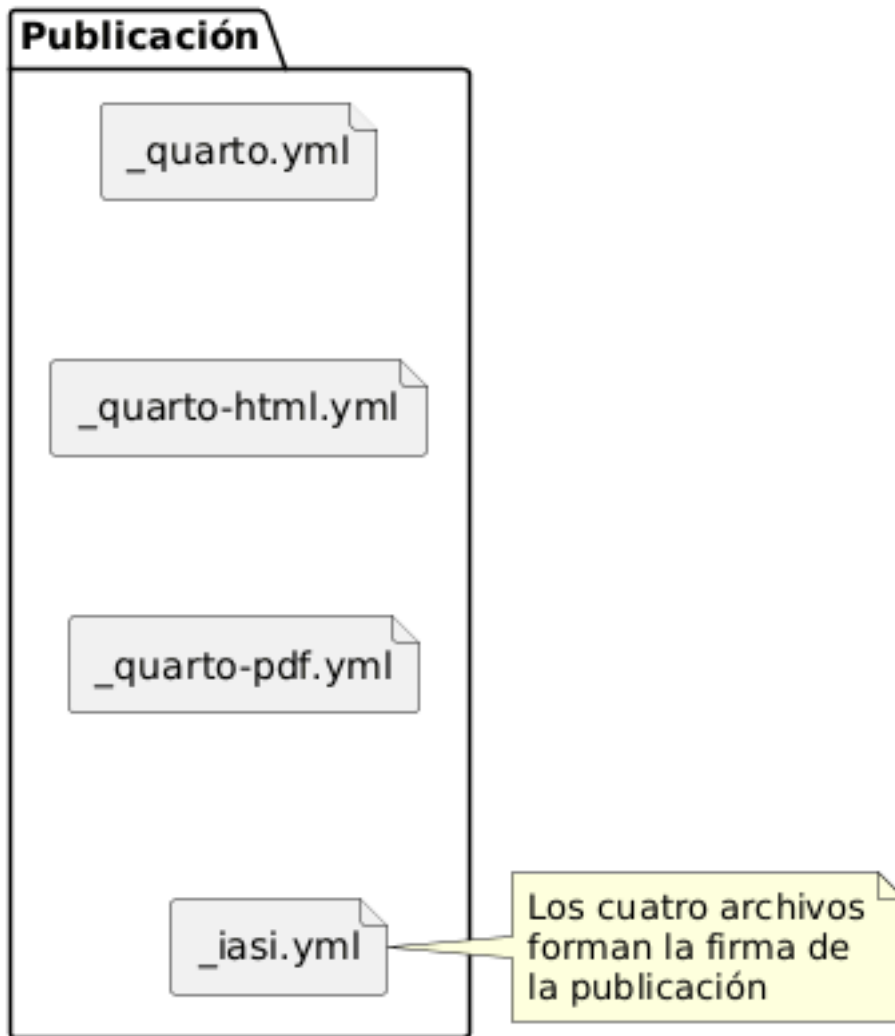
Una publicación IASI Quarto sigue siendo un proyecto Quarto.

`iasi.quarto` no sustituye su estructura ni introduce un contenedor documental propio. Añade una convención mínima que permite descubrir la publicación, comprobarla, preparar sus artefactos derivados y delegar finalmente el renderizado en Quarto.

La estructura mínima de una publicación es:

```
publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  _iasi.yml
```

La presencia conjunta de estos cuatro archivos identifica una publicación IASI Quarto.



4.3 Configuración general y configuración por formato

El archivo `_quarto.yml` contiene la configuración común de Quarto:

```
project:
  type: book

profile:
  group:
    - [html, pdf]
```

```
metadata-files:
- _book-structure.yml
```

Las configuraciones específicas de cada salida se mantienen separadas:

```
_quarto-html.yml
  configuración de la publicación HTML

_quarto-pdf.yml
  configuración de la publicación PDF
```

Esta separación permite que una misma publicación produzca distintos formatos sin mezclar sus decisiones particulares.

Por ejemplo, el HTML puede utilizar una hoja de estilos CSS, mientras que el PDF puede declarar opciones específicas de LaTeX.

4.4 Configuración de `iasi.quarto`

El archivo `_iasi.yml` describe cómo debe interpretar `iasi.quarto` el contenido de la publicación.

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Sus valores predeterminados son:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Por tanto, una publicación puede omitir las propiedades que no necesite modificar.

`_iasi.yml` no contiene configuración de renderizado. Su responsabilidad es describir la organización editorial que `iasi.quarto` debe descubrir y preparar.

4.5 Directorio de contenido

La propiedad `content-dir` señala dónde se encuentran los documentos de la publicación.

```
publication:
  content-dir: chapters
```

Con esta configuración, la estructura habitual es:

```
publication/
  chapters/
    _quarto.yml
    _quarto-html.yml
    _quarto-pdf.yml
    _iasi.yml
```

El nombre `chapters` es el valor predeterminado, pero no forma parte rígida del modelo.

También sería válida una publicación como:

```
publication/
  content/
    _quarto.yml
    _quarto-html.yml
    _quarto-pdf.yml
    _iasi.yml
```

si `_iasi.yml` declara:

```
publication:
  content-dir: content
```

4.6 Documentos de raíz

Los documentos situados en la raíz de la publicación pertenecen directamente al libro.

Por ejemplo:

```
publication/  
  index.qmd  
  manifesto.qmd  
  principles.qmd  
  licenses.qmd  
  chapters/
```

`iasi.quarto` conserva estos documentos y los incorpora antes o después del contenido organizado en carpetas, según su posición dentro de la estructura preparada.

El `index.qmd` de la raíz es siempre un documento del autor.

No debe confundirse con los archivos `index.qmd` que pueden existir dentro del directorio de contenido.

4.7 Carpetas de contenido

Cada carpeta inmediata de `content-dir` representa una unidad editorial.

```
chapters/  
  01-fundamentals/  
  02-installation/  
  03-usage/
```

El significado exacto de cada carpeta depende de la estrategia de publicación.

En una publicación **regular**, cada carpeta se convierte en un capítulo compuesto.

En una publicación **structured**, cada carpeta se convierte en una parte que contiene capítulos independientes.

La estructura física puede ser parecida, pero su interpretación editorial es distinta.

4.8 Estructura regular

Una carpeta **regular** contiene documentos parciales que se ensamblan en un único capítulo.

```
chapters/  
  01-fundamentals/  
    front-matter.quarto  
    00-index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

El archivo `front-matter.quarto` es opcional.

Cuando existe, contiene un front matter Quarto completo:

```
---  
title: "Fundamentals"  
title-block-style: none  
---
```

`iasi.quarto` lo copia literalmente al principio del `index.qmd` generado.

Los documentos `.qmd` contienen el cuerpo del capítulo:

```
## La publicación  
  
Contenido de la sección.
```

Durante la preparación, `iasi.quarto` genera:

```
chapters/  
  01-fundamentals/  
    front-matter.quarto  
    00-index.qmd  
    01-publication.qmd  
    02-project-structure.qmd  
    index.qmd
```

El resultado derivado tiene esta forma:

```
---  
title: "Fundamentals"  
title-block-style: none  
---  
  
<!-- Generated by iasi.quarto. Do not edit. -->
```

```
{{{< include 00-index.qmd >}}}  
{{{< include 01-publication.qmd >}}}  
{{{< include 02-project-structure.qmd >}}}
```

En esta estrategia:

```
el autor mantiene los documentos parciales  
iasi.quarto mantiene el index.qmd de la carpeta
```

Si `front-matter.quarto` no existe, el capítulo se genera sin front matter. Quarto podrá mostrar `index.html` como nombre de navegación, haciendo visible que falta la metadata editorial.

4.9 Estructura structured

En una publicación **structured**, cada carpeta contiene capítulos independientes.

```
chapters/  
  01-fundamentals/  
    index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

El `index.qmd` de la carpeta pertenece al autor y actúa como introducción de la parte.

Los demás documentos son capítulos completos y comienzan normalmente con un encabezado de primer nivel:

```
# Estructura del proyecto
```

En esta estrategia, `iasi.quarto` no genera ni reemplaza el `index.qmd` de la carpeta.

```
regular:  
  iasi.quarto mantiene el index.qmd  
  
structured:  
  el autor mantiene el index.qmd
```

4.10 Archivos fuente y archivos derivados

Una publicación contiene dos clases de archivos.

Los archivos fuente son mantenidos por el autor:

```
_iasi.yml
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
front-matter.quarto
*.qmd
```

Los archivos derivados son producidos por `iasi.quarto`:

```
_book-structure.yml
index.qmd de las carpetas regular
```

Esta separación es importante.

Los archivos derivados pueden regenerarse en cualquier momento. No deben utilizarse como lugar de edición porque su contenido pertenece al proceso de preparación.



4.11 La estructura preparada

Quarto necesita una relación explícita de los capítulos que forman el libro.

`iasi.quarto` genera esa relación en:

```
_book-structure.yml
```

Por ejemplo:

```
book:
  chapters:
    - "index.qmd"
    - "chapters/01-fundamentals/index.qmd"
    - "chapters/02-installation/index.qmd"
```


El archivo `_quarto.yml` incorpora esta estructura mediante:

```
metadata-files:  
  - _book-structure.yml
```

De esta forma, la configuración principal permanece estable mientras la estructura editorial puede regenerarse automáticamente.

4.12 Publicaciones múltiples

Un directorio superior puede contener varias publicaciones independientes:

```
workspace/  
  01-user-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    _iasi.yml  
  
  02-developer-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    _iasi.yml
```

El directorio `workspace` no necesita ser una publicación ni contener un `_iasi.yml`.

`iasi.quarto` descubre recursivamente las carpetas que contienen la firma completa y trata cada una como una unidad de construcción independiente.

4.13 Una estructura explícita

La estructura de una publicación IASI Quarto no pretende ocultar Quarto.

Todos sus elementos continúan siendo visibles:

```
los documentos siguen siendo .qmd  
la configuración sigue siendo YAML  
los perfiles siguen perteneciendo a Quarto  
los formatos finales siguen siendo generados por Quarto
```

`iasi.quarto` añade una capa de organización y preparación, pero mantiene abiertas las piezas que forman la publicación.

La estructura no reemplaza el proyecto Quarto. Lo hace reconocible, comprobable y reproducible.

4.14 El archivo `_iasi.yml`

`_iasi.yml` describe cómo debe interpretar `iasi.quarto` una publicación.

No sustituye a `_quarto.yml` ni contiene opciones de renderizado. Su función es declarar la organización editorial que `iasi.quarto` debe descubrir, comprobar y preparar antes de delegar la generación final en Quarto.

La estructura actual es:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Las tres propiedades son opcionales. Cuando no se declaran, `iasi.quarto` utiliza sus valores predeterminados.

4.15 Valores predeterminados

La configuración mínima válida es:

```
publication:
```

También puede escribirse simplemente:

```
publication: {}
```

En ambos casos, el resultado efectivo es:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Por tanto, estas dos configuraciones son equivalentes:

```
publication: {}
```

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Los valores predeterminados permiten crear una publicación convencional sin repetir configuración que ya forma parte del modelo.

4.16 strategy

La propiedad **strategy** indica cómo debe interpretarse el contenido de cada carpeta inmediata del directorio de contenido.

```
publication:
  strategy: regular
```

Los valores admitidos son:

```
regular
structured
direct
```

4.16.1 regular

En la estrategia **regular**, cada carpeta representa un capítulo compuesto por varios documentos parciales.

```
chapters/
  01-fundamentals/
    front-matter.quarto
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

Durante la preparación, `iasi.quarto` genera:

```
chapters/01-fundamentals/index.qmd
```

Ese archivo reúne los documentos parciales mediante inclusiones Quarto.

```
{{< include 00-index.qmd >}}  
  
{{< include 01-publication.qmd >}}  
  
{{< include 02-project-structure.qmd >}}
```

En esta estrategia, el `index.qmd` de la carpeta es un artefacto derivado y pertenece a `iasi.quarto`.

4.16.2 structured

En la estrategia **structured**, cada carpeta representa una parte y sus documentos son capítulos independientes.

```
chapters/  
  01-fundamentals/  
    index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

El `index.qmd` de la carpeta pertenece al autor y actúa como introducción de la parte.

Los demás documentos se incorporan como capítulos independientes dentro de esa parte.

4.16.3 direct

La estrategia **direct** conserva una relación más directa entre los documentos encontrados y la estructura entregada a Quarto.

No genera capítulos compuestos y no asume que cada carpeta deba convertirse en una parte o en un capítulo ensamblado.

Se utiliza cuando la publicación ya controla explícitamente su organización o cuando no encaja en las convenciones de **regular** y **structured**.

4.17 content-dir

La propiedad `content-dir` indica dónde se encuentra el contenido organizado de la publicación.

```
publication:
  content-dir: chapters
```

El valor predeterminado es:

```
chapters
```

Con esa configuración, `iasi.quarto` busca las carpetas editoriales en:

```
publication/
  chapters/
```

Puede utilizarse otro nombre:

```
publication:
  content-dir: content
```

La publicación correspondiente podría ser:

```
publication/
  content/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  _iasi.yml
```

`content-dir` se interpreta como una ruta relativa a la raíz de la publicación.

No es una ruta relativa al directorio desde el que se ejecuta R ni al directorio de trabajo del usuario.

4.18 numbered

La propiedad `numbered` indica si las carpetas y los documentos fuente deben seguir una convención de prefijos numéricos.

```
publication:
  numbered: true
```

Cuando su valor es `true`, las carpetas deben comenzar por uno o más dígitos seguidos de un guion:

```
01-fundamentals
02-installation
10-reference
```

El patrón utilizado es:

```
^[0-9]+-
```

Los documentos fuente `.qmd` deben seguir la misma convención:

```
00-index.qmd
01-publication.qmd
02-project-structure.qmd
```

El patrón utilizado es:

```
^[0-9]+-.*\.qmd$
```

Los prefijos determinan el orden editorial porque el descubrimiento se apoya en el orden de los nombres.

Por ejemplo:

```
01-introduction.qmd
02-model.qmd
10-reference.qmd
```

se procesa en ese mismo orden.

4.18.1 `numbered: false`

Una publicación puede desactivar esta exigencia:

```
publication:
  numbered: false
```

En ese caso, `iasi.quarto` no exige prefijos numéricos en carpetas ni documentos.

```
chapters/
  fundamentals/
  installation/
  usage/
```

y:

```
introduction.qmd
publication.qmd
project-structure.qmd
```

siguen siendo nombres válidos.

Desactivar la validación no añade un mecanismo alternativo de ordenación. La publicación continúa dependiendo del orden lexicográfico de los nombres encontrados.

4.19 Configuración parcial

No es necesario declarar todas las propiedades.

Por ejemplo:

```
publication:
  strategy: structured
```

produce esta configuración efectiva:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

Del mismo modo:

```
publication:
  numbered: false
```

mantiene `regular` como estrategia y `chapters` como directorio de contenido.

`iasi.quarto` completa únicamente las propiedades ausentes. No sustituye valores que hayan sido declarados explícitamente.

4.20 Configuración válida completa

Una publicación regular convencional puede declarar:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Una publicación estructurada puede declarar:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

Una publicación sin prefijos numéricos podría utilizar:

```
publication:
  strategy: regular
  content-dir: content
  numbered: false
```

4.21 YAML mal formado

`_iasi.yml` debe ser un documento YAML válido.

Por ejemplo, esta configuración es incorrecta:

```
publication:
  strategy regular
```


Falta el separador `:` entre la propiedad y su valor.

Los errores sintácticos se detectan durante el descubrimiento, cuando `iasi.quarto` intenta leer el archivo.

La publicación no puede continuar hacia la comprobación ni hacia la preparación mientras el YAML no pueda interpretarse.

4.22 Valores semánticamente inválidos

Un documento puede ser YAML válido y, aun así, contener una configuración no admitida.

Por ejemplo:

```
publication:
  strategy: automatic
```

`automatic` es una cadena válida desde el punto de vista de YAML, pero no es una estrategia reconocida por `iasi.quarto`.

También es inválido:

```
publication:
  numbered: yes
```

Aunque algunas implementaciones históricas de YAML pueden interpretar `yes` como un valor lógico, el contrato de `iasi.quarto` espera una configuración inequívoca y debe expresarse como:

```
publication:
  numbered: true
```

o:

```
publication:
  numbered: false
```

Los valores semánticamente inválidos se rechazan durante la fase de comprobación.

4.23 Propiedades desconocidas

El contrato de `_iasi.yml` es explícito.

Esta configuración contiene una propiedad desconocida:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
  language: es
```

`language` no pertenece al modelo de `iasi.quarto`.

El idioma de la publicación debe configurarse en Quarto:

```
lang: es
```

dentro de `_quarto.yml`.

La presencia de propiedades desconocidas se considera un error porque puede ocultar erratas o configuraciones que el autor cree activas cuando en realidad no tienen ningún efecto.

Por ejemplo:

```
publication:
  stratgy: regular
```

no debe ignorarse silenciosamente. La propiedad está mal escrita y la publicación debe informar del problema.

4.24 Separación de responsabilidades

Cada archivo de configuración tiene una responsabilidad distinta.

```
_quarto.yml
  configuración común de Quarto

_quarto-html.yml
  configuración específica de HTML

_quarto-pdf.yml
```

```
configuración específica de PDF

_iasi.yml
  organización editorial interpretada por iasi.quarto
```

Por ejemplo, esta opción pertenece a `_quarto-html.yml`:

```
format:
  html:
    theme: cosmo
```

Esta pertenece a `_quarto-pdf.yml`:

```
format:
  pdf:
    toc: true
```

Y esta pertenece a `_iasi.yml`:

```
publication:
  strategy: regular
```

Mantener esta separación evita que `iasi.quarto` se convierta en una segunda capa de configuración de Quarto.

4.25 Configuración y ciclo de construcción

`_iasi.yml` participa en las primeras fases del proceso:

```
validate()
  reconoce la publicación por su firma

.discover()
  lee _iasi.yml y aplica valores predeterminados

.check()
  comprueba propiedades, valores y estructura

.prepare()
  construye los artefactos derivados según la estrategia
```

La configuración no se interpreta durante el renderizado.

Cuando `.render()` se ejecuta, la publicación ya ha sido descubierta, comprobada y preparada.



4.26 Un contrato pequeño

`_iasi.yml` contiene deliberadamente pocas propiedades.

No pretende describir toda la publicación ni duplicar la configuración disponible en Quarto.

Su contrato actual responde únicamente a tres preguntas:

```
¿Cómo se interpreta el contenido?  
  strategy  
  
¿Dónde se encuentra?  
  content-dir  
  
¿Debe seguir una convención numérica?  
  numbered
```

Esa limitación mantiene la configuración legible y permite que la mayor parte del proyecto siga siendo Quarto abierto y reconocible.

`_iasi.yml` no describe cómo se renderiza una publicación. Describe cómo debe entenderse antes de renderizarla.

4.27 Estrategias de publicación

La estrategia de una publicación determina cómo interpreta `iasi.quarto` las carpetas y documentos situados dentro de `content-dir`.

Se declara en `_iasi.yml`:

```
publication:  
  strategy: regular
```

El contrato actual admite tres estrategias:

```
regular
structured
direct
```

La estrategia se aplica a la publicación completa. No describe el formato de salida y no cambia entre HTML y PDF.

Su responsabilidad es editorial:

```
strategy
  determina cómo se transforma la estructura fuente
  en la estructura que Quarto debe renderizar
```

4.28 Una misma estructura física, distintas publicaciones

Dos publicaciones pueden contener carpetas y documentos parecidos, pero producir estructuras editoriales diferentes.

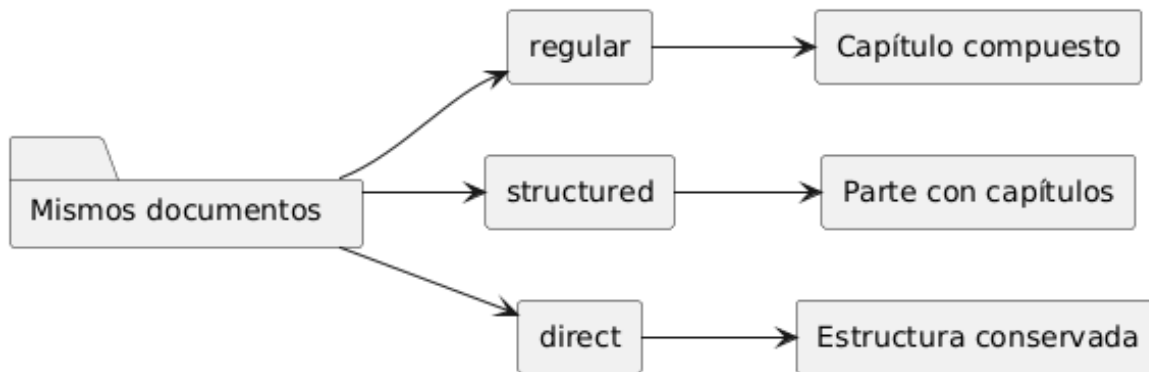
Por ejemplo:

```
chapters/
  01-fundamentals/
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

Con estrategia **regular**, esos documentos forman un único capítulo compuesto.

Con estrategia **structured**, cada documento puede convertirse en un capítulo independiente dentro de una parte.

La diferencia no reside únicamente en los nombres de archivo. Reside en quién controla la composición editorial y en qué artefactos debe preparar `iasi.quarto`.



4.29 Estrategia regular

`regular` es la estrategia predeterminada.

```
publication:  
  strategy: regular
```

Está pensada para redactar un capítulo mediante varios documentos parciales y pequeños. Cada carpeta inmediata de `content-dir` representa un capítulo.

```
chapters/  
  01-fundamentals/  
  02-installation/  
  03-usage/
```

Dentro de cada carpeta, los documentos `.qmd` representan secciones del capítulo:

```
chapters/  
  01-fundamentals/  
    front-matter.quarto  
    00-index.qmd  
    01-publication.qmd  
    02-project-structure.qmd  
    03-iasi-yml.qmd
```

Los documentos parciales comienzan normalmente en nivel dos:

```
## La publicación
```

```
Contenido de la sección.
```

No necesitan declarar un título de capítulo en nivel uno porque el capítulo se construye mediante el `index.qmd` generado.

4.29.1 Propiedad del `index.qmd`

En regular, el `index.qmd` de cada carpeta pertenece a `iasi.quarto`.

No debe mantenerse manualmente.

Durante `.prepare()`, el sistema genera un archivo con las inclusiones ordenadas:

```
<!-- Generated by iasi.quarto. Do not edit. -->

{{< include 00-index.qmd >}}

{{< include 01-publication.qmd >}}

{{< include 02-project-structure.qmd >}}

{{< include 03-iasi-yml.qmd >}}
```

El archivo generado es un artefacto derivado y puede recrearse en cualquier momento.

```
autor
  mantiene los documentos parciales

iasi.quarto
  mantiene el index.qmd compuesto
```

4.29.2 `front-matter.quarto`

Una carpeta regular puede contener:

```
front-matter.quarto
```

Este archivo es opcional y contiene un front matter Quarto completo:

```

---
title: "Fundamentals"
title-block-style: none
toc: true
---

```

iasi.quarto no interpreta cada propiedad por separado ni inventa valores que falten.

Copia el contenido literalmente al comienzo del `index.qmd` generado.

```

---
title: "Fundamentals"
title-block-style: none
toc: true
---

<!-- Generated by iasi.quarto. Do not edit. -->

{{< include 00-index.qmd >}}

```

Si el archivo no existe, el capítulo se genera sin front matter.

4.29.3 Estructura resultante

La estructura fuente:

```

chapters/
  01-fundamentals/
    front-matter.quarto
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd

```

se convierte durante la preparación en:

```

chapters/
  01-fundamentals/
    front-matter.quarto
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd
    index.qmd

```


Y `_book-structure.yml` incorpora únicamente el capítulo compuesto:

```
book:
  chapters:
    - "index.qmd"
    - "chapters/01-fundamentals/index.qmd"
```

Los documentos parciales no aparecen como capítulos independientes en la estructura del libro.

4.29.4 Cuándo utilizar regular

regular es apropiada cuando:

```
un capítulo crece demasiado para mantenerse en un único archivo
se desea dividir la escritura sin dividir el capítulo editorial
las secciones deben compartir el mismo título y front matter
el autor prefiere documentos pequeños y ensamblables
```

El modelo permite separar la unidad de escritura de la unidad de publicación.

```
varios archivos fuente
  forman
un capítulo publicado
```

4.30 Estrategia structured

structured está pensada para libros en los que cada carpeta representa una parte y cada documento es un capítulo independiente.

```
publication:
  strategy: structured
```

La estructura habitual es:

```
chapters/
  01-fundamentals/
    index.qmd
    01-publication.qmd
    02-project-structure.qmd
    03-iasi-yml.qmd
```

En esta estrategia:

```
la carpeta
  representa una parte

index.qmd
  presenta la parte

cada otro .qmd
  representa un capítulo
```

4.30.1 Propiedad del index.qmd

En `structured`, el `index.qmd` de la carpeta pertenece al autor.

`iasi.quarto` no lo genera ni lo reemplaza.

El archivo puede contener la introducción de la parte:

```
# Fundamentals

Esta parte presenta el modelo fundamental de una publicación IASI Quarto.
```

Los demás documentos son capítulos completos y comienzan normalmente con un encabezado de nivel uno:

```
# La publicación

Contenido del capítulo.
```

La regla de propiedad es inversa a la estrategia `regular`:

```
regular
  iasi.quarto mantiene index.qmd

structured
  el autor mantiene index.qmd
```

4.30.2 Estructura resultante

La estructura fuente:

```
chapters/  
  01-fundamentals/  
    index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

produce una estructura editorial equivalente a:

```
book:  
  chapters:  
    - "index.qmd"  
    - part: "chapters/01-fundamentals/index.qmd"  
      chapters:  
        - "chapters/01-fundamentals/01-publication.qmd"  
        - "chapters/01-fundamentals/02-project-structure.qmd"
```

La carpeta no se ensambla en un único documento.

Cada capítulo conserva su identidad dentro de la parte.

4.30.3 front-matter.quarto no participa

front-matter.quarto pertenece al modelo de composición de regular.

En structured, el autor controla directamente el front matter de cada documento:

```
---  
title: "La publicación"  
---  
  
# La publicación
```

No se necesita un archivo externo para construir el `index.qmd` porque ese archivo ya es fuente del autor.

4.30.4 Cuándo utilizar structured

structured es apropiada cuando:

```
el libro se organiza explícitamente en partes
cada documento debe aparecer como capítulo independiente
cada capítulo necesita su propio título o metadata
el autor quiere controlar el index.qmd introductorio de cada parte
```

El modelo coincide de forma directa con la estructura editorial clásica de un libro Quarto.

```
una carpeta
  contiene
una parte y sus capítulos
```

4.31 Estrategia direct

direct es la estrategia mínima.

```
publication:
  strategy: direct
```

Su objetivo es reducir al mínimo la transformación aplicada por `iasi.quarto`.

No construye capítulos compuestos como `regular` y no impone el modelo de partes de `structured`.

Los documentos descubiertos se conservan como unidades editoriales directas.

Por ejemplo:

```
chapters/
  01-introduction.qmd
  02-configuration.qmd
  03-reference.qmd
```

puede trasladarse a una estructura equivalente:

```
book:
  chapters:
    - "index.qmd"
    - "chapters/01-introduction.qmd"
    - "chapters/02-configuration.qmd"
    - "chapters/03-reference.qmd"
```

También puede utilizarse con carpetas cuando la publicación ya contiene documentos completos y no necesita que `iasi.quarto` genere índices de composición.

4.31.1 Qué no hace `direct`

La estrategia `direct` no debe entenderse como una estrategia que adivina la intención del autor.

No:

```
combina documentos parciales
crea títulos de capítulo
genera front matter
convierte carpetas automáticamente en partes
```

Su papel es conservar la estructura encontrada con la menor intervención posible.

4.31.2 Cuándo utilizar `direct`

`direct` es apropiada cuando:

```
la publicación ya posee documentos completos
la organización no encaja en regular ni structured
se desea utilizar iasi.quarto para validar y construir
sin introducir una composición editorial adicional
```

Es también una salida útil para publicaciones heredadas o estructuras deliberadamente simples.

4.32 Comparación

Las tres estrategias pueden resumirse así:

Estrategia	Carpeta inmediata	Documentos .qmd	index.qmd de carpeta	Preparación principal
regular	Capítulo compuesto	Secciones parciales	Generado por <code>iasi.quarto</code>	Ensamblar inclusiones
structured	Parte	Capítulos independientes	Mantenido por el autor	Construir parte y capítulos
direct	Contenedor opcional	Unidades directas	No se genera por convención	Conservar la estructura

La diferencia esencial es la unidad editorial que representa cada archivo.

```
regular
  varios archivos = un capítulo

structured
  varios archivos = varios capítulos dentro de una parte

direct
  cada archivo conserva su papel directo
```

4.33 Estrategia y numeración

La propiedad `numbered` es independiente de `strategy`.

Por ejemplo:

```
publication:
  strategy: regular
  numbered: false
```

permite:

```
chapters/
  fundamentals/
    introduction.qmd
    publication.qmd
    project-structure.qmd
```

Y:

```
publication:
  strategy: structured
  numbered: true
```

exige:

```
chapters/
  01-fundamentals/
    index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

`strategy` define el significado editorial.

`numbered` define la convención de nombres y orden.

4.34 Estrategia y formatos de salida

La estrategia tampoco depende del formato final.

Una publicación puede construirse como HTML y PDF:

```
build()
```

o únicamente como HTML:

```
build(format = "html")
```

En ambos casos se utiliza la misma estructura preparada.

```
fuentes
-> estrategia
-> estructura editorial
-> HTML

fuentes
-> estrategia
-> estructura editorial
-> PDF
```

No existe una estrategia `regular` para HTML y otra `structured` para PDF.

La organización editorial se resuelve antes de renderizar los formatos.

4.35 Elección de estrategia

La elección puede reducirse a tres preguntas.

```
¿Varios archivos deben formar un solo capítulo?  
  regular  
  
¿Cada carpeta debe ser una parte con capítulos propios?  
  structured  
  
¿La estructura debe conservarse con mínima transformación?  
  direct
```

Cambiar de estrategia no es únicamente cambiar una opción.

Puede cambiar:

```
el significado de las carpetas  
el nivel de los encabezados  
la propiedad de los index.qmd  
la estructura generada para Quarto
```

Por eso la estrategia debe elegirse según la unidad editorial deseada, no según la comodidad momentánea de los nombres de archivo.

4.36 Una decisión editorial explícita

Las estrategias no intentan inferir qué quiso decir el autor.

Lo declaran.

```
publication:  
  strategy: regular
```

convierte una convención implícita en un contrato verificable.

`iasi.quarto` puede entonces descubrir la estructura, comprobarla y generar únicamente los artefactos que corresponden a esa decisión.

La estrategia no define dónde están los archivos. Define qué significan.

4.37 Ciclo de construcción

`iasi.quarto` organiza la construcción de una publicación como una secuencia de etapas explícitas.

La entrada pública principal es:

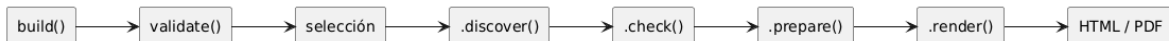
```
build()
```

Internamente, la construcción sigue este flujo:

```
build()
-> validate()
-> seleccionar publicaciones
-> .discover()
-> .check()
-> .prepare()
-> resolver formatos
-> .render()
```

Cada etapa tiene una responsabilidad concreta.

No todas modifican archivos y no todas necesitan conocer los mismos detalles de la publicación.



4.38 `build()` como orquestador

`build()` es la operación pública que coordina el proceso completo.

```
build()
```

Puede construir:

```
una publicación actual
varias publicaciones dentro de un multiproyecto
todos los formatos soportados
un formato seleccionado
```

Por ejemplo:

```
build(format = "html")
```

o:

```
build(  
  book = "user-guide",  
  format = "pdf"  
)
```

`build()` no concentra toda la lógica en una única función.

Delega cada decisión en la etapa que corresponde:

```
reconocer el espacio de trabajo  
  validate()  
  
leer la configuración  
  .discover()  
  
comprobar el contrato  
  .check()  
  
generar artefactos derivados  
  .prepare()  
  
invocar Quarto  
  .render()
```

Esto permite que el proceso falle en el punto correcto y que cada etapa pueda entenderse de forma independiente.

4.39 Primera etapa: `validate()`

`validate()` reconoce si una ruta contiene una publicación IASI Quarto o un multiproyecto que puede contener varias.

```
plan = validate(".")
```

La función busca la firma de publicación:

```
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
_iasi.yml
```

Si los cuatro archivos existen en la ruta actual, esa ruta se considera una publicación.

Si no existen juntos, `validate()` puede reconocer la ruta como un contenedor desde el que después se localizarán publicaciones completas.

El resultado es un plan:

```
class(plan)
```

```
"iasi_quarto_plan"  
"list"
```

El plan contiene inicialmente:

```
plan$path  
plan$current  
plan$books
```

4.39.1 Publicación actual

Cuando la propia ruta validada contiene la firma completa:

```
plan$current = TRUE
```

La publicación actual tiene prioridad.

No se buscan publicaciones anidadas porque la ruta ya representa una unidad de construcción completa.

4.39.2 Multiproyecto

Cuando la ruta no es una publicación, pero contiene publicaciones descendientes:

```
plan$current = FALSE
```

El plan actúa como contenedor de las publicaciones que podrán seleccionarse y descubrirse.

`validate()` reconoce el terreno.

Todavía no interpreta `_iasi.yml` ni genera artefactos.

4.40 Selección de publicaciones

En un multiproyecto, la selección ocurre después de `validate()` y antes de `.discover()`.

```
validate()  
-> seleccionar publicaciones  
-> .discover()
```

Este orden es importante.

Si el usuario solicita una publicación concreta:

```
build(book = "user-guide")
```

solo esa publicación debe continuar por el pipeline.

Una publicación no seleccionada no debe bloquear la construcción aunque contenga una configuración incompleta o inválida.

La selección admite:

```
nombre completo del directorio  
nombre sin prefijo numérico  
prefijo numérico
```

Por ejemplo, una carpeta:

```
01-user-guide
```

puede seleccionarse mediante:

```
build(book = "01-user-guide")
```

```
build(book = "user-guide")
```

```
build(book = "1")
```

En una publicación actual, el argumento `book` no es necesario porque la unidad de construcción ya está determinada.

4.41 Segunda etapa: `.discover()`

`.discover()` transforma el plan validado en un modelo enriquecido de las publicaciones seleccionadas.

```
plan = .discover(plan)
```

Durante esta fase se leen:

```
_quarto.yml  
_iasi.yml
```

y se determina:

```
nombre de la publicación  
ruta  
tipo Quarto  
estrategia  
directorio de contenido  
uso de numeración  
configuración efectiva
```

Cada publicación descubierta se representa mediante un proyecto:

```
project$name  
project$path  
project$quarto_file  
project$iasi_file  
project$type
```

```
project$strategy
project$content_dir
project$content_path
project$numbered
project$quarto
project$iasi
```

La clase del objeto es:

```
iasiquarto_project
```

4.41.1 Aplicación de valores predeterminados

`.discover()` completa las propiedades ausentes de `_iasi.yml`.

Por ejemplo:

```
publication:
  strategy: structured
```

se descubre como:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

La etapa distingue entre:

```
configuración ausente
  se completa con valores predeterminados
```

```
YAML ilegible
  detiene el descubrimiento
```

Un archivo que no puede interpretarse como YAML no puede convertirse en un proyecto descubierto.

4.42 Tercera etapa: `.check()`

`.check()` verifica que las publicaciones descubiertas respeten el contrato de `iasi.quarto`.

```
plan = .check(plan)
```

La comprobación se ocupa de aspectos semánticos y estructurales.

Por ejemplo:

```
estrategia soportada  
content-dir válido  
numbered lógico  
propiedades conocidas  
convenciones de nombres  
estructura coherente con la estrategia
```

Un YAML puede estar bien formado y, aun así, ser incorrecto:

```
publication:  
  strategy: automatic
```

`.discover()` puede leerlo, pero `.check()` debe rechazarlo porque `automatic` no es una estrategia soportada.

4.42.1 Resultado de la comprobación

Cada proyecto recibe un estado:

```
project$valid
```

El plan resume el resultado:

```
plan$valid
```

La construcción solo continúa si todas las publicaciones seleccionadas son válidas.

```
plan$valid = TRUE
-> continúa hacia .prepare()

plan$valid = FALSE
-> se detiene antes de generar artefactos
```

Esta separación evita que una publicación incorrecta llegue hasta Quarto y produzca errores más tardíos y menos precisos.

4.43 Cuarta etapa: `.prepare()`

`.prepare()` genera la estructura derivada que Quarto necesita para renderizar la publicación.

```
plan = .prepare(plan)
```

Esta es la primera etapa que puede escribir archivos.

Los artefactos generados dependen de la estrategia.

4.43.1 Preparación regular

En una publicación regular, `.prepare()` genera el `index.qmd` de cada carpeta editorial.

```
chapters/01-fundamentals/index.qmd
```

Ese archivo:

```
copia front-matter.quarto si existe
añade la cabecera de archivo generado
incluye los documentos parciales en orden
```

También genera:

```
_book-structure.yml
```

con los capítulos compuestos que Quarto debe procesar.

4.43.2 Preparación structured

En una publicación `structured`, `.prepare()` conserva los `index.qmd` mantenidos por el autor.

No los reemplaza.

Su trabajo principal consiste en construir `_book-structure.yml` con:

```
partes
introducciones de parte
capítulos independientes
```

4.43.3 Preparación direct

En una publicación `direct`, `.prepare()` aplica la mínima transformación necesaria.

No construye capítulos compuestos ni convierte automáticamente cada carpeta en una parte.

4.43.4 Artefactos

La publicación preparada mantiene información sobre los archivos derivados:

```
publication$chapters
publication$artifacts
publication$changed
```

`publication$changed` indica si la preparación tuvo que modificar algún artefacto.

4.44 Idempotencia de la preparación

La preparación debe ser idempotente.

Esto significa que ejecutar dos veces el mismo proceso sobre las mismas fuentes no debe reescribir continuamente los mismos archivos.

```
build()
build()
```

Si nada ha cambiado:

```
primera ejecución
  puede generar artefactos

segunda ejecución
  encuentra el mismo contenido
  no vuelve a escribirlo
```

La escritura se realiza únicamente cuando el contenido nuevo es distinto del existente.

Esto reduce:

```
cambios innecesarios en Git
marcas de tiempo artificiales
reconstrucciones provocadas por archivos idénticos
ruido en el proceso
```

La idempotencia convierte los artefactos derivados en resultados reproducibles, no en residuos de cada ejecución.

4.45 Resolución de formatos

Después de preparar la publicación, `build()` determina los formatos que deben renderizarse.

Los formatos soportados actualmente son:

```
html
pdf
```

Cuando no se indica ninguno:

```
build()
```

se construyen todos los formatos soportados.

También puede elegirse uno:

```
build(format = "html")
```

o:

```
build(format = "pdf")
```

La resolución de formatos pertenece al orquestador público.

`.render()` recibe un único formato ya resuelto y no vuelve a validarlo.

```
build()  
  decide qué formatos  
  
.render()  
  ejecuta un formato
```

4.46 Quinta etapa: `.render()`

`.render()` es una operación privada y mínima.

Recibe:

```
una publicación preparada  
un formato resuelto
```

Su contrato conceptual es:

```
.render(publication, format)
```

Por ejemplo:

```
publicación 01-user-guide  
formato html
```

representa una ejecución.

La misma publicación en PDF representa otra:

```
publicación 01-user-guide  
formato pdf
```

En un multiproyecto con dos publicaciones y dos formatos, `build()` coordina cuatro renderizados:

```
01-user-guide      -> html
01-user-guide      -> pdf
02-developer-guide -> html
02-developer-guide -> pdf
```

4.46.1 Delegación en Quarto

`.render()` no implementa un motor documental propio.

Invoca Quarto con el perfil y formato correspondientes:

```
quarto render --profile html --to html
```

o:

```
quarto render --profile pdf --to pdf
```

La publicación ya debe estar validada, descubierta, comprobada y preparada.

`.render()` no vuelve hacia atrás para resolver problemas de configuración o estructura.

4.46.2 Resultado

Después de un renderizado correcto, la publicación registra:

```
publication$rendered
publication$profiles
```

Por ejemplo:

```
publication$rendered = TRUE
publication$profiles = c("html", "pdf")
```

4.47 Fallar antes de renderizar

Una propiedad importante del ciclo es que los errores se detecten lo antes posible.

```
ruta no reconocible
  validate()

YAML ilegible
  .discover()

configuración o estructura inválida
  .check()

problema al generar artefactos
  .prepare()

fallo real de Quarto
  .render()
```

Cada problema pertenece a una etapa.

Esto evita utilizar el renderizado como mecanismo genérico de validación.

Quarto debe recibir una publicación ya preparada, no una colección de decisiones pendientes.

4.48 Ciclo de una publicación individual

Para una publicación actual:

```
publication/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  _iasi.yml
```

el flujo es:

```
build()
  reconoce la publicación
  descubre su configuración
  comprueba su estructura
  prepara sus artefactos
  renderiza HTML y PDF
```

No existe selección entre publicaciones porque la ruta actual ya es la unidad de construcción.

4.49 Ciclo de un multiproyecto

Para:

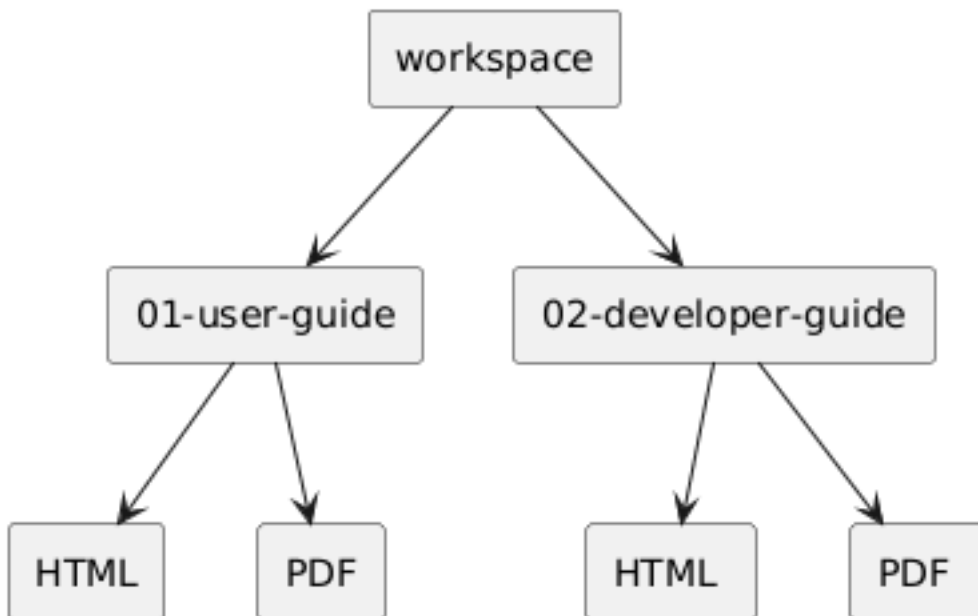
```
workspace/  
  01-user-guide/  
  02-developer-guide/
```

el flujo es:

```
build()  
  reconoce el workspace  
  selecciona publicaciones  
  descubre solo las seleccionadas  
  comprueba cada una  
  prepara cada una  
  renderiza cada combinación de publicación y formato
```

La unidad de trabajo de `.render()` sigue siendo una publicación y un formato.

El multiproyecto solo amplía la orquestación.



4.50 Estado acumulado

El plan se enriquece durante el proceso.

```
validate()
  crea el plan

.discover()
  añade proyectos y configuración

.check()
  añade el resultado de comprobación

.prepare()
  añade publicaciones y artefactos

.render()
  añade formatos renderizados
```

El resultado final de `build()` no es únicamente un código de salida.

Es el plan completado, con información sobre lo que se descubrió, preparó y renderizó.

```
plan = build()
```

Aunque `build()` lo devuelve de forma invisible, el objeto puede conservarse para inspección.

4.51 Un pipeline deliberado

El ciclo de construcción separa reconocimiento, interpretación, comprobación, preparación y ejecución.

```
validate
  ¿es un espacio IASI Quarto?

discover
  ¿qué contiene y cómo está configurado?

check
  ¿es coherente con el contrato?
```

```
prepare
  ¿qué artefactos necesita Quarto?

render
  ¿puede Quarto producir el formato solicitado?
```

Cada etapa reduce incertidumbre antes de pasar a la siguiente.

`build()` no es un único salto desde los fuentes hasta el PDF. Es una cadena de decisiones comprobables.

5 Instalación

5.1 Instalación

`iasi.quarto` se distribuye como un paquete de R.

Su instalación añade al entorno dos operaciones públicas:

```
validate()  
build()
```

La primera reconoce una publicación IASI Quarto. La segunda ejecuta su ciclo completo de construcción.

El paquete no sustituye a Quarto ni instala por sí mismo sus herramientas externas. Trabaja sobre un entorno en el que R y Quarto ya están disponibles.



La instalación se divide en tres pasos:

```
comprobar los requisitos  
instalar iasi.quarto  
verificar el entorno
```

En primer lugar, se comprueba que R y Quarto puedan ejecutarse correctamente.

Después se instala el paquete desde su repositorio.

Por último, se realiza una comprobación mínima para confirmar que `iasi.quarto` puede cargarse y reconocer una publicación.

Este capítulo no pretende explicar cómo instalar R o Quarto en cada sistema operativo. Su objetivo es dejar preparado el entorno específico que necesita `iasi.quarto`.

Al finalizar, debe ser posible ejecutar:

```
library(iasi.quarto)

validate()
```

y obtener un plan cuando el directorio actual contiene una publicación o un multiproyecto IASI Quarto.

La instalación prepara la herramienta. La estructura y la configuración de la publicación siguen perteneciendo al proyecto.

5.2 Requisitos

`iasi.quarto` necesita dos componentes para funcionar:

```
R
Quarto
```

R ejecuta el paquete.

Quarto procesa la publicación y genera los formatos finales.

Para construir PDF también se necesita una distribución de TeX disponible para Quarto.



5.3 R

`iasi.quarto` es un paquete de R y debe instalarse en una biblioteca accesible desde la sesión que ejecutará la construcción.

Puede comprobarse la instalación de R desde la consola:

```
R.version.string
```

El resultado debe mostrar la versión activa de R.

También conviene comprobar dónde se instalarán y buscarán los paquetes:

```
.libPaths()
```

No es necesario utilizar RStudio.

El paquete puede ejecutarse desde:

```
RStudio  
la consola de R  
Rscript  
un proceso automatizado  
un pipeline de integración continua
```

RStudio es un entorno de trabajo posible, no una dependencia de `iasi.quarto`.

5.4 Quarto

`iasi.quarto` delega el renderizado final en la línea de comandos de Quarto.

Por tanto, el ejecutable `quarto` debe estar instalado y disponible para el proceso de R.

Desde una terminal:

```
quarto --version
```

Desde R:

```
Sys.which("quarto")
```

El resultado de `Sys.which()` debe contener una ruta.

Por ejemplo:

```
quarto  
"C:\Program Files\Quarto\bin\quarto.exe"
```

Si devuelve una cadena vacía:

```
quarto  
""
```

R no puede localizar el ejecutable mediante la variable `PATH`.

También puede comprobarse la instalación completa desde una terminal:

```
quarto check
```

Esta orden informa sobre Quarto, Pandoc y los componentes auxiliares disponibles.

5.5 HTML

La construcción HTML necesita:

```
R  
iasi.quarto  
Quarto
```

No requiere una distribución de TeX.

Una vez instalado el paquete, el formato puede comprobarse con:

```
build(format = "html")
```

5.6 PDF

La construcción PDF añade una dependencia externa:

```
una distribución de TeX
```

Puede utilizarse, por ejemplo:

```
TinyTeX  
TeX Live  
MiKTeX
```

`iasi.quarto` no instala ni administra esa distribución.

Quarto debe poder localizarla y utilizarla durante el renderizado.

La comprobación práctica es:

```
build(format = "pdf")
```

Si HTML funciona y PDF no, el primer elemento que debe revisarse es el entorno TeX, no la estructura de `iasi.quarto`.

5.7 Git

Git no es una dependencia de ejecución.

`iasi.quarto` puede cargar y construir una publicación sin que Git esté instalado.

Sin embargo, Git resulta útil para:

```
obtener el código desde un repositorio  
actualizar el paquete  
versionar las publicaciones  
trabajar con ramas y entregas
```

Su disponibilidad puede comprobarse desde una terminal:

```
git --version
```

La necesidad real de Git depende del método de instalación utilizado.

5.8 Acceso al repositorio

Mientras `iasi.quarto` se distribuya desde un repositorio de código, la instalación necesita acceso a ese repositorio.

El acceso puede realizarse mediante:

```
HTTPS  
una copia local  
un archivo del paquete
```

Una vez instalado, el paquete no necesita consultar el repositorio para ejecutar `validate()` o `build()`.

5.9 Permisos de escritura

Durante la preparación, `iasi.quarto` puede generar archivos dentro de la publicación:

```
_book-structure.yml  
index.qmd de las carpetas regular
```

La cuenta que ejecuta la construcción debe tener permiso de escritura sobre el proyecto.

También debe poder escribir en los directorios de salida y temporales utilizados por Quarto.

Una publicación de solo lectura puede validarse, pero no podrá completar una preparación que necesite crear o actualizar artefactos.

5.10 Red y servicios externos

La construcción básica de `iasi.quarto` no necesita un servicio remoto propio.

Sin embargo, una publicación concreta puede utilizar recursos externos:

```
repositorios de paquetes  
fuentes web  
bibliografía remota  
extensiones Quarto  
servidores de diagramas
```

Por ejemplo, el filtro PlantUML puede depender de un servidor configurado por la publicación.

Esas dependencias pertenecen al proyecto documental o a sus extensiones, no al núcleo de `iasi.quarto`.

5.11 Comprobación mínima

Antes de instalar el paquete, el entorno puede revisarse con:

```
R.version.string  
Sys.which("quarto")
```

Y desde una terminal:

```
quarto --version  
quarto check
```

Para construir ambos formatos, la comprobación mínima es:

```
R disponible  
Quarto disponible  
TeX disponible para PDF  
permisos de escritura sobre la publicación
```

No se necesita un entorno especial ni un editor concreto.

`iasi.quarto` organiza la construcción, pero utiliza las herramientas reales de R y Quarto que ya existen en el sistema.

5.12 Instalar `iasi.quarto`

`iasi.quarto` se distribuye actualmente desde su repositorio de GitHub:

```
iasi-org/iasi-quarto
```

El nombre del repositorio utiliza un guion:

```
iasi-quarto
```

El nombre del paquete de R utiliza un punto:

```
iasi.quarto
```

Esta diferencia aparece en la instalación y en el uso:

```
remotes::install_github("iasi-org/iasi-quarto")  
  
library(iasi.quarto)
```

5.13 Instalar remotes

La instalación desde GitHub puede realizarse con el paquete `remotes`.

Si todavía no está instalado:

```
install.packages("remotes")
```

Esta operación solo es necesaria una vez por biblioteca de R.

Puede comprobarse su disponibilidad con:

```
requireNamespace(  
  "remotes",  
  quietly = TRUE  
)
```

El resultado esperado es:

```
TRUE
```

5.14 Instalar la versión actual

Para instalar la versión disponible en la rama principal del repositorio:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

`remotes` descarga el código fuente, resuelve las dependencias declaradas por el paquete y ejecuta la instalación en la biblioteca activa de R.

Una vez terminada:

```
library(iasi.quarto)
```

La carga debe completarse sin errores.

5.15 Instalar una versión concreta

Una instalación reproducible debe indicar la versión que necesita.

Por ejemplo, para instalar la versión 0.5.0:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@v0.5.0"  
)
```

La referencia situada después de @ puede ser:

```
una etiqueta  
una rama  
un commit
```

Una etiqueta publicada es la opción natural para una versión estable:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@v0.5.0"  
)
```

Un commit permite fijar exactamente el código utilizado:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@<commit>"  
)
```

La instalación sin referencia:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

utiliza el estado actual de la rama predeterminada y puede cambiar con el tiempo.

5.16 Actualizar el paquete

Para actualizar una instalación existente, se ejecuta de nuevo la instalación:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

No es necesario desinstalar previamente el paquete.

Cuando `iasi.quarto` ya está cargado en la sesión, R puede mantener archivos del paquete en uso, especialmente en Windows.

En ese caso:

```
reinicie la sesión de R  
vuelva a ejecutar la instalación  
cargue de nuevo el paquete
```

En RStudio, la sesión puede reiniciarse desde:

```
Session  
-> Restart R
```

Después:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)  
  
library(iasi.quarto)
```

5.17 Comprobar la versión instalada

La versión activa puede consultarse con:

```
packageVersion("iasi.quarto")
```

Para la versión 0.5.0, el resultado será equivalente a:

```
[1] '0.5.0'
```

También puede consultarse la descripción completa del paquete:

```
packageDescription("iasi.quarto")
```

La ruta desde la que se ha cargado puede obtenerse con:

```
find.package("iasi.quarto")
```

Esta comprobación resulta útil cuando existen varias bibliotecas de R y no está claro cuál contiene la instalación activa.

5.18 Biblioteca de instalación

R instala el paquete en una de las rutas declaradas por:

```
.libPaths()
```

La primera ruta suele ser la biblioteca utilizada por defecto.

Puede consultarse antes de instalar:

```
.libPaths()
```

Y comprobar después dónde quedó instalado:

```
find.package("iasi.quarto")
```

Para instalar expresamente en una biblioteca concreta:

```
remotes::install_github(  
  "iasi-org/iasi-quarto",  
  lib = "path/to/library"  
)
```

La misma ruta debe estar disponible en `.libPaths()` cuando se cargue el paquete.

5.19 Dependencias

Las dependencias declaradas por `iasi.quarto` se instalan durante el proceso cuando no están disponibles.

La instalación no incorpora:

```
Quarto CLI
una distribución de TeX
servicios externos utilizados por la publicación
```

Esos componentes pertenecen al entorno de construcción y deben estar preparados por separado.

El paquete puede instalarse correctamente aunque todavía falte TeX. En ese caso, podrá cargarse y construir HTML, pero el renderizado PDF no estará completo hasta que Quarto disponga de una distribución de TeX.

5.20 Instalación desde una copia local

Cuando ya existe una copia local del repositorio, puede instalarse directamente desde su raíz.

Con remotes:

```
remotes::install_local(
  "path/to/iasi-quarto"
)
```

Durante el desarrollo del propio paquete también puede utilizarse:

```
devtools::install(
  "path/to/iasi-quarto"
)
```

La ruta debe señalar el directorio que contiene el archivo:

```
DESCRIPTION
```

La instalación local utiliza el estado actual de los archivos, incluidos los cambios que todavía no se hayan publicado en GitHub.

5.21 Cargar el paquete

La instalación coloca el paquete en una biblioteca de R.

Para utilizarlo en una sesión:

```
library(iasi.quarto)
```

Esto incorpora su API pública:

```
validate()  
build()
```

También puede llamarse una función sin cargar todo el paquete:

```
iasi.quarto::validate()
```

```
iasi.quarto::build()
```

Esta forma hace explícito el paquete al que pertenece cada función.

5.22 Instalación reproducible

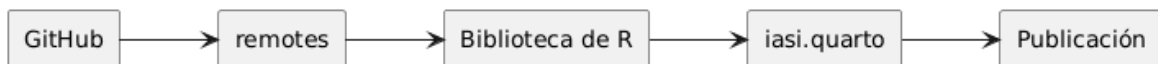
Para un entorno personal de desarrollo puede ser suficiente instalar la rama principal:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

Para una construcción que deba repetirse en otro equipo o en integración continua, debe fijarse una versión:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@v0.5.0"  
)
```

De este modo, el código de la publicación y la herramienta que interpreta su estructura evolucionan de forma controlada.



5.23 Instalación mínima

La secuencia mínima completa es:

```
install.packages("remotes")

remotes::install_github(
  "iasi-org/iasi-quarto@v0.5.0"
)

library(iasi.quarto)

packageVersion("iasi.quarto")
```

La instalación termina cuando R puede cargar el paquete desde su biblioteca.

La siguiente comprobación consiste en verificar que `iasi.quarto` reconoce una publicación real y puede ejecutar su ciclo de construcción.

5.24 Verificar la instalación

La instalación queda verificada cuando R puede cargar `iasi.quarto`, localizar Quarto y reconocer una publicación válida.

La comprobación puede realizarse en cuatro pasos:

```
comprobar el paquete
comprobar Quarto
validar una publicación
construir un formato
```

5.25 Comprobar el paquete

Cargue el paquete:

```
library(iasi.quarto)
```

La operación debe terminar sin errores.

Después, consulte la versión instalada:

```
packageVersion("iasi.quarto")
```

Para la versión 0.5.0, el resultado será equivalente a:

```
[1] '0.5.0'
```

También puede comprobarse la ruta desde la que R ha cargado el paquete:

```
find.package("iasi.quarto")
```

Esto resulta útil cuando existen varias bibliotecas de R o varias instalaciones del mismo paquete.

5.26 Comprobar Quarto

`iasi.quarto` necesita que el ejecutable de Quarto esté disponible para la sesión de R.

Compruébelo con:

```
Sys.which("quarto")
```

El resultado debe contener una ruta.

Por ejemplo:

```
quarto  
"C:\Program Files\Quarto\bin\quarto.exe"
```

Si el resultado es una cadena vacía:

```
quarto  
""
```

R no puede localizar Quarto mediante la variable `PATH`.

También puede comprobarse la versión desde R:

```
system2(  
  "quarto",  
  "--version"  
)
```

Y desde una terminal:

```
quarto --version
```

5.27 Situarse en una publicación

Para verificar `validate()`, la sesión debe trabajar sobre una publicación IASI Quarto o sobre un multiproyecto que contenga publicaciones completas.

Una publicación mínima contiene:

```
publication/  
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
_iasi.yml
```

Desde R puede comprobarse el directorio actual con:

```
getwd()
```

Cuando la publicación se encuentra en otra ruta:

```
setwd(  
  "path/to/publication"  
)
```

También puede evitarse el cambio de directorio pasando la ruta explícitamente:

```
plan = validate(  
  "path/to/publication"  
)
```

5.28 Validar la publicación

Ejecute:


```
plan = validate()
```

Si el directorio actual contiene una publicación o multiproyecto IASI Quarto, `validate()` devuelve invisiblemente un plan.

Puede comprobarse con:

```
is.null(plan)
```

El resultado esperado es:

```
FALSE
```

Y su clase:

```
class(plan)
```

debe incluir:

```
"iasi_quarto_plan"
```

También pueden inspeccionarse sus propiedades iniciales:

```
plan$path  
plan$current  
plan$books
```

5.28.1 Publicación actual

Cuando la ruta validada contiene la firma completa:

```
plan$current
```

debe devolver:

```
TRUE
```

5.28.2 Multiproyecto

Cuando la ruta actúa como contenedor de varias publicaciones:

```
plan$current
```

debe devolver:

```
FALSE
```

Las publicaciones candidatas quedan registradas en:

```
plan$books
```

5.29 Directorio no reconocido

Si la ruta no contiene una publicación ni un multiproyecto IASI Quarto:

```
plan = validate(  
  "path/to/ordinary-directory"  
)
```

el resultado es:

```
NULL
```

Esto no indica un fallo de instalación.

Indica que la ruta no cumple la firma de publicación esperada.

La diferencia es importante:

```
library(iasi.quarto) falla  
  problema de instalación del paquete  
  
Sys.which("quarto") devuelve ""  
  Quarto no está disponible para R  
  
validate() devuelve NULL  
  la ruta no es un espacio IASI Quarto
```

5.30 Construir HTML

La comprobación funcional mínima consiste en construir HTML:

```
build(  
  format = "html"  
)
```

HTML es la primera prueba recomendada porque no depende de una distribución de TeX.

La construcción completa recorre:

```
validate()  
.discover()  
.check()  
.prepare()  
.render()
```

Si termina correctamente, quedan verificados:

```
la carga del paquete  
la lectura de la configuración  
la estructura de la publicación  
la generación de artefactos  
la invocación de Quarto
```

5.31 Inspeccionar el resultado

`build()` devuelve invisiblemente el plan completado.

Puede conservarse:

```
plan = build(  
  format = "html"  
)
```

Después pueden inspeccionarse:

```
plan$formats  
plan$rendered  
plan$elapsed
```

El formato esperado es:

```
plan$formats
```

```
"html"
```

Y, después de un renderizado correcto:

```
plan$rendered
```

debe ser:

```
TRUE
```

Cada proyecto contiene también su publicación preparada:

```
publication = plan$projects[[1]]$publication
```

Pueden consultarse:

```
publication$chapters  
publication$artifacts  
publication$changed  
publication$rendered  
publication$profiles
```

5.32 Construir PDF

Una vez verificado HTML, puede comprobarse PDF:

```
build(  
  format = "pdf"  
)
```

Esta prueba añade la dependencia de TeX.

Si HTML funciona y PDF falla, el paquete y la estructura básica ya han superado la verificación principal.

En ese caso, debe revisarse:

```
la instalación de TeX  
la integración de TeX con Quarto  
las dependencias LaTeX de la publicación
```

La comprobación general puede realizarse desde una terminal:

```
quarto check
```

5.33 Verificación completa

La secuencia completa es:

```
library(iasiq.quarto)  
  
packageVersion("iasiq.quarto")  
  
Sys.which("quarto")  
  
plan = validate()  
  
stopifnot(  
  !is.null(plan)  
)  
  
html = build(  
  format = "html"  
)  
  
stopifnot(  
  isTRUE(html$rendered)  
)
```

Después, cuando el entorno PDF esté preparado:

```
pdf = build(  
  format = "pdf"  
)  
  
stopifnot(  
  isTRUE(pdf$rendered)  
)
```

5.34 Diagnóstico rápido

Los fallos más habituales pueden localizarse por la etapa en la que aparecen.

Comprobación	Resultado	Interpretación
<code>library(iasi.quarto)</code>	error	El paquete no está instalado o no está disponible en <code>.libPaths()</code>
<code>Sys.which("quarto")</code> <code>validate()</code>	<code>""</code> NULL	R no puede localizar Quarto La ruta no contiene una publicación IASI Quarto
<code>build(format = "html")</code>	error antes de Quarto	La configuración o estructura no supera el pipeline
<code>build(format = "html")</code>	error de Quarto	Quarto no pudo renderizar la publicación
<code>build(format = "pdf")</code>	falla con HTML correcto	Debe revisarse el entorno TeX

```
PlantUML version 1.2026.6 / 6287b33 [2026-06-08 16:13:24 UTC]

This version of PlantUML is 218 days old, so you should
consider upgrading from https://plantuml.com/download

[From string (line 4) ]

@startuml
top to bottom direction

start
Syntax Error? (Assumed diagram type: class)
```

5.35 Resultado esperado

La instalación puede considerarse correcta cuando:

```
R carga iasi.quarto
Quarto está disponible
validate() reconoce la publicación
build(format = "html") termina correctamente
```

PDF puede verificarse después como una capacidad adicional del entorno.

La instalación no termina al copiar el paquete. Termina cuando una publicación real atraviesa el pipeline y llega a Quarto.

6 Workflow recomendado

El trabajo habitual con `iasi.quarto` separa la edición, la construcción y la publicación. Esta separación permite revisar los resultados antes de reemplazar los artefactos que se desplegarán.

El recorrido recomendado es:

```
editar
-> validar
-> construir HTML
-> revisar
-> construir todos los formatos
-> publicar
-> desplegar
```

6.1 1. Trabajar desde la raíz adecuada

Sitúese en la publicación que desea construir:

```
setwd("path/to/publication")
```

También puede trabajar desde la raíz de un multiproyecto. En ese caso, utilice `book` para seleccionar una publicación cuando no quiera procesarlas todas.

6.2 2. Validar el espacio de trabajo

Antes de construir, compruebe que la ruta se reconoce correctamente:

```
library(iasi.quarto)

plan = validate()
```

En una publicación individual:


```
plan$current
```

debe ser TRUE. En un multiproyecto, `plan$books` muestra las publicaciones encontradas.

`validate()` reconoce el espacio de trabajo, pero no genera documentos ni sustituye una construcción completa.

6.3 3. Construir HTML durante la edición

HTML suele ofrecer el ciclo de comprobación más rápido:

```
build(format = "html")
```

En un multiproyecto puede limitarse la operación:

```
build(  
  path = ".",  
  book = "user-guide",  
  format = "html"  
)
```

Después de cada cambio significativo, revise al menos:

- la navegación;
- la estructura de capítulos;
- los enlaces internos;
- las figuras y diagramas;
- los bloques de código;
- los avisos producidos durante la construcción.

6.4 4. Construir todos los formatos

Cuando el contenido HTML sea correcto, construya todos los formatos configurados:

```
build()
```

En un libro con perfiles HTML y PDF, esta llamada genera ambos formatos. Un HTML correcto no garantiza por sí solo que el PDF pueda generarse: el entorno PDF necesita además una instalación TeX funcional.

Los resultados permanecen en los directorios declarados por los perfiles, normalmente bajo `_outputs/`. Esta etapa no reemplaza todavía `publish/`.

6.5 5. Revisar los artefactos construidos

Antes de publicar, compruebe los resultados generados:

```
_outputs/  
  html/  
  pdf/
```

La revisión debe confirmar que:

- la portada y el índice del PDF son correctos;
- el HTML incluye sus recursos y navegación;
- los enlaces entre HTML y PDF tendrán sentido en el resultado publicado;
- no faltan imágenes, diagramas ni hojas de estilo.

6.6 6. Ensamblar publish/

Cuando los resultados sean válidos, ensamble la publicación:

```
publish()
```

`publish()` no vuelve a renderizar. Utiliza los artefactos ya construidos y crea el directorio desplegable `publish/`.

En un multiproyecto, reúne las publicaciones seleccionadas bajo una estructura común. También puede seleccionar una publicación concreta:

```
publish(book = "user-guide")
```

6.7 7. Utilizar deploy() al cerrar el ciclo

Cuando quiera construir y publicar en una única operación:

```
deploy()
```

`deploy()` ejecuta primero `build()` y solo llama a `publish()` si la construcción termina correctamente. Si el build falla, conserva el `publish/` anterior.

Esta operación es apropiada cuando los fuentes están listos para convertirse en el nuevo artefacto publicado. Durante la edición diaria suele ser preferible ejecutar `build(format = "html")` de forma independiente.

6.8 Ciclo resumido

Para desarrollo:

```
validate()  
build(format = "html")
```

Para revisión completa:

```
build()
```

Para ensamblar resultados ya comprobados:

```
publish()
```

Para cerrar el ciclo en una sola operación:

```
deploy()
```

Construir sirve para comprobar. Publicar sirve para ensamblar. Desplegar coordina ambas operaciones cuando el contenido está listo.

7 API pública

La API pública de `iasi.quarto` es deliberadamente pequeña.

Actualmente expone cuatro funciones:

```
validate()
build()
publish()
deploy()
```

El resto del ciclo de construcción permanece como implementación interna:

```
.discover()
.check()
.prepare()
.render()
```

Esta separación establece una frontera clara entre:

```
lo que utiliza quien publica
lo que coordina internamente el paquete
```

La API pública no reproduce cada fase del pipeline como una operación independiente. Ofrece los puntos de entrada necesarios para reconocer, construir, publicar y desplegar una publicación.

7.1 `validate()`

`validate()` reconoce una publicación IASI Quarto o un multiproyecto.

```
plan = validate()
```

También puede recibir una ruta explícita:

```
plan = validate("path/to/workspace")
```

Su firma conceptual es:

```
validate(path = ".")
```

La función comprueba la presencia de la firma IASI Quarto:

```
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
_iasi.yml
```

Cuando los cuatro archivos están presentes en la ruta indicada, esa ruta se reconoce como una publicación actual.

Cuando no están presentes juntos, la ruta puede reconocerse como un contenedor desde el que posteriormente se descubrirán publicaciones completas.

7.2 Resultado de `validate()`

Cuando la ruta corresponde a un espacio IASI Quarto, `validate()` devuelve invisiblemente un plan:

```
class(plan)
```

```
"iasi_quarto_plan"  
"list"
```

El plan inicial contiene:

```
plan$path  
plan$current  
plan$books
```

7.2.1 `plan$path`

Contiene la ruta normalizada utilizada como raíz del proceso.

```
plan$path
```

Representa la publicación actual o el multiproyecto validado.

7.2.2 plan\$current

Indica si la propia ruta es una publicación completa:

```
plan$current
```

TRUE

la ruta actual contiene la firma completa

FALSE

la ruta actúa como contenedor de publicaciones

Cuando `current` es TRUE, la publicación actual tiene prioridad y no se buscan publicaciones anidadas.

7.2.3 plan\$books

Contiene las publicaciones candidatas localizadas desde un multiproyecto.

En una publicación actual, la unidad de construcción ya está determinada y no es necesario seleccionar entre varios libros.

7.3 Ruta no reconocida

Cuando la ruta no parece una publicación ni un multiproyecto IASI Cuarto, `validate()` devuelve:

```
NULL
```

Por ejemplo:

```
plan = validate("empty-directory")
```

```
is.null(plan)
```

TRUE

Esto permite utilizar `validate()` como operación de reconocimiento sin iniciar una construcción completa.

7.4 Qué hace `validate()`

`validate()` responde a una pregunta limitada:

¿La ruta parece contener un espacio de trabajo IASI Quarto?

No:

```
interpreta por completo _iasi.yml
comprueba toda la estructura editorial
genera index.qmd
genera _book-structure.yml
invoca Quarto
```

Esas tareas pertenecen a etapas posteriores del pipeline.

La función valida la frontera exterior del espacio de trabajo, no toda su semántica interna.

7.5 Ejemplo con una publicación

Dada esta estructura:

```
user-guide/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  _iasi.yml
```

la llamada:

```
plan = validate("user-guide")
```

produce conceptualmente:

```
plan$current = TRUE
plan$path    = ".../user-guide"
```

7.6 Ejemplo con un multiproyecto

Dada esta estructura:

```
workspace/
  01-user-guide/
    _quarto.yml
    _quarto-html.yml
    _quarto-pdf.yml
    _iasi.yml

  02-developer-guide/
    _quarto.yml
    _quarto-html.yml
    _quarto-pdf.yml
    _iasi.yml
```

la llamada:

```
plan = validate("workspace")
```

produce un plan de multiproyecto:

```
plan$current = FALSE
```

y conserva las publicaciones candidatas para la selección posterior.

7.7 build()

`build()` es el punto de entrada principal del paquete.

```
build()
```

Su firma pública es:


```
build(  
    book = NULL,  
    format = NULL,  
    path = "."  
)
```

La función ejecuta el ciclo completo:

```
validate()  
selección de publicaciones  
.discover()  
.check()  
.prepare()  
.render()
```

build() puede operar sobre:

```
la publicación actual  
todas las publicaciones de un multiproyecto  
una publicación seleccionada  
todos los formatos soportados  
un formato concreto
```

7.8 Construcción predeterminada

La llamada más sencilla es:

```
build()
```

Cuando se ejecuta dentro de una publicación:

```
construye la publicación actual  
genera HTML  
genera PDF
```

Cuando se ejecuta en un multiproyecto:

```
construye todas las publicaciones descubiertas  
genera todos los formatos soportados
```

Los formatos soportados actualmente son:

```
html  
pdf
```

7.9 Argumento path

`path` indica la publicación o multiproyecto que debe construirse.

```
build(path = "path/to/publication")
```

Su valor predeterminado es:

```
path = "."
```

Por tanto, `build()` trabaja normalmente sobre el directorio actual.

Ejemplos:

```
build(path = "01-user-guide")
```

```
build(path = "workspace")
```

La ruta se valida antes de iniciar las demás etapas.

Si no corresponde a un espacio IASI Quarto, `build()` devuelve invisiblemente:

```
NULL
```

7.10 Argumento format

`format` selecciona el formato o formatos de salida.

Puede utilizarse:

```
build(format = "html")
```

```
build(format = "pdf")
```

También puede indicarse:

```
build(format = "all")
```

Los valores:

```
NULL
```

y:

```
"all"
```

seleccionan todos los formatos soportados.

Conceptualmente:

```
format = NULL  
  -> html y pdf  
  
format = "all"  
  -> html y pdf  
  
format = "html"  
  -> solo html  
  
format = "pdf"  
  -> solo pdf
```

Un formato no soportado produce un error antes del renderizado:

```
build(format = "docx")
```

```
Unsupported format: "docx".
```

La validación de formatos pertenece a `build()`.

`.render()` recibe siempre un formato ya resuelto.

7.11 Argumento book

book selecciona una o varias publicaciones dentro de un multiproyecto.

Por ejemplo:

```
build(book = "user-guide")
```

La selección admite tres formas.

7.11.1 Nombre completo

```
build(book = "01-user-guide")
```

7.11.2 Nombre sin prefijo numérico

```
build(book = "user-guide")
```

7.11.3 Prefijo numérico

```
build(book = "1")
```

Las tres expresiones pueden seleccionar:

```
01-user-guide
```

También pueden solicitarse varias publicaciones:

```
build(  
  book = c(  
    "user-guide",  
    "developer-guide"  
  )  
)
```

7.12 `book = NULL` y `book = "all"`

Cuando `book` no se indica:

```
build(book = NULL)
```

se construyen todas las publicaciones disponibles en el multiproyecto.

También puede expresarse:

```
build(book = "all")
```

En una publicación actual, la selección mediante `book` no es necesaria.

La ruta ya identifica la única publicación que debe construirse.

7.13 Selecciones no encontradas

Cuando una selección no existe, `build()` informa mediante una advertencia.

Por ejemplo:

```
build(  
  book = c(  
    "user-guide",  
    "missing"  
  )  
)
```

puede continuar construyendo `user-guide` y advertir:

```
Books not found: "missing".
```

Una publicación no encontrada no se inventa ni se sustituye por otra coincidencia aproximada.

7.14 Selección antes del descubrimiento

La selección se aplica antes de `.discover()`.

```
validate()  
  -> selección  
  -> .discover()
```

Esto tiene una consecuencia importante.

Una publicación no seleccionada no participa en:

```
lectura de _iasi.yml  
comprobación  
preparación  
renderizado
```

Por tanto, una publicación experimental o incompleta no bloquea la construcción de otra publicación seleccionada.

7.15 Ejemplos de uso

7.15.1 Construir la publicación actual

```
build()
```

7.15.2 Construir solo HTML

```
build(format = "html")
```

7.15.3 Construir solo PDF

```
build(format = "pdf")
```

7.15.4 Construir un multiproyecto completo

```
build(path = "workspace")
```

7.15.5 Construir una publicación del multiproyecto

```
build(  
  path = "workspace",  
  book = "user-guide"  
)
```

7.15.6 Construir una publicación y un formato

```
build(  
  path = "workspace",  
  book = "user-guide",  
  format = "html"  
)
```

7.15.7 Construir varias publicaciones

```
build(  
  path = "workspace",  
  book = c(  
    "1",  
    "developer-guide"  
  ),  
  format = "pdf"  
)
```

7.16 Resultado de build()

build() devuelve invisiblemente el plan completado:

```
plan = build()
```

El objeto conserva información acumulada durante el pipeline.

Por ejemplo:

```
plan$path  
plan$current  
plan$projects  
plan$valid  
plan$formats  
plan$rendered  
plan$elapsed
```

Cada proyecto puede contener su publicación preparada:

```
plan$projects[[1]]$publication
```

y esta puede registrar:

```
publication$chapters  
publication$artifacts  
publication$changed  
publication$rendered  
publication$profiles
```

La devolución invisible permite dos formas de uso.

Uso habitual:

```
build()
```

Uso con inspección:

```
plan = build()  
  
plan$formats  
plan$elapsed
```

7.17 Mensajes de construcción

Aunque el plan se devuelve invisiblemente, `build()` informa del proceso mediante mensajes.

La salida puede mostrar:


```
publicaciones reconocidas
estado de comprobación
formatos construidos
resultado final
tiempo empleado
```

Los mensajes están dirigidos a la persona que ejecuta la construcción.

El objeto devuelto está dirigido a la inspección programática.

7.18 publish()

publish() ensambla en publish/ los artefactos que ya fueron construidos:

```
publish()
```

Su firma pública es:

```
publish(
  book = NULL,
  path = "."
)
```

La función no renderiza de nuevo. Comprueba la publicación, localiza los directorios de salida declarados por los perfiles y copia los resultados necesarios al destino publicable.

En un multiproyecto puede seleccionarse una o varias publicaciones:

```
publish(
  path = "workspace",
  book = "user-guide"
)
```

El resultado devuelve invisiblemente el plan publicado y registra, entre otros datos:

```
plan$published
plan$publish_path
plan$elapsed
```

Debe llamarse después de una construcción correcta. Si faltan los artefactos esperados, publish() informa del problema en lugar de presentar una publicación incompleta como válida.

7.19 deploy()

`deploy()` coordina la construcción y la publicación:

```
deploy()
```

Su comportamiento es equivalente a:

```
build()  
  -> si termina correctamente  
publish()
```

Si `build()` falla, `publish()` no se ejecuta y el directorio `publish/` anterior queda intacto.

`deploy()` no realiza operaciones Git ni publica por sí mismo en un servicio remoto. En la API de `iasi.quarto`, desplegar significa preparar el artefacto local `publish/` mediante un ciclo completo y seguro.

7.20 API pública y funciones privadas

Las fases internas no forman parte del contrato público:

```
.discover()  
.check()  
.prepare()  
.render()
```

El prefijo `.` expresa que son funciones privadas del paquete.

No deben utilizarse como puntos de entrada estables desde código externo.

Por ejemplo, esto pertenece a la implementación:

```
plan = .discover(plan)
```

La forma pública de ejecutar el proceso es:

```
build()
```

La diferencia no es meramente estética.

Las funciones privadas pueden cambiar su estructura interna mientras se conserva el contrato de:

```
validate()  
build()
```

7.21 Por qué no se expone cada etapa

Una API con todas las fases públicas obligaría a quien la utiliza a conocer:

```
el orden correcto  
los tipos intermedios  
las precondiciones de cada función  
los cambios internos del pipeline
```

Por ejemplo, no tendría sentido ejecutar `.prepare()` sobre un plan que todavía no ha pasado por `.discover()` y `.check()`.

`build()` conserva esas precondiciones dentro del paquete.

```
usuario  
  llama a build()  
  
build()  
  garantiza el orden correcto
```

Esto reduce la superficie pública sin ocultar el modelo conceptual.

Las etapas siguen existiendo y pueden documentarse, pero no se convierten en obligaciones para quien utiliza el paquete.

7.22 Elegir la operación pública

Cada función pública corresponde a una intención distinta.

```
validate()
    reconoce y devuelve un plan inicial

build()
    ejecuta la construcción completa

publish()
    ensambla artefactos ya construidos en publish/

deploy()
    ejecuta build() y, si termina correctamente, publish()
```

Utilice `validate()` cuando necesite comprobar qué representa una ruta:

```
plan = validate("workspace")
```

Utilice `build()` cuando necesite generar la publicación:

```
build("workspace")
```

Con argumentos nombrados, la llamada correcta es:

```
build(path = "workspace")
```

porque el primer argumento de `build()` es `book`, no `path`.

7.23 Llamadas con argumentos nombrados

Para evitar ambigüedades, es recomendable nombrar los argumentos cuando se proporciona una ruta:

```
build(path = "workspace")
```

y no:

```
build("workspace")
```

La segunda expresión se interpreta como:

```
book = "workspace"
```

porque la firma es:

```
build(  
    book = NULL,  
    format = NULL,  
    path = "."  
)
```

Del mismo modo:

```
build(  
    book = "user-guide",  
    format = "html",  
    path = "workspace"  
)
```

hace explícita cada decisión.

7.24 Contrato estable, implementación evolutiva

La API pública protege al usuario de la evolución interna del paquete.

El pipeline puede incorporar en el futuro:

```
nuevas comprobaciones  
nuevos artefactos  
nuevos formatos  
nuevas estrategias
```

sin exigir que el código externo coordine manualmente cada cambio.

La frontera se mantiene pequeña:

```
validate()  
build()  
publish()  
deploy()
```

La API pública expresa la intención. El pipeline privado resuelve el procedimiento.